

Time Series Forecasting using ML

BITS Pilani-Hyderabad

Dr. Tanujit Chakraborty

Sorbonne University, Abu Dhabi, UAE

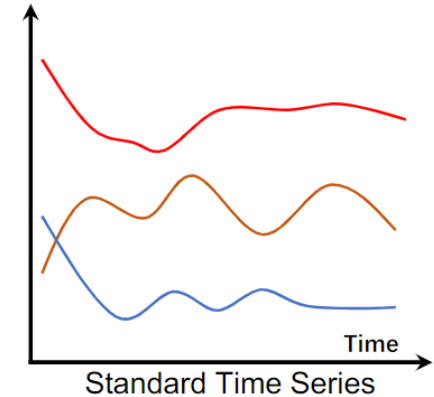
Sorbonne Centre for Artificial Intelligence, Paris, France

Research Areas: Time Series Forecasting, Machine Learning, Econometrics, Health Data Science

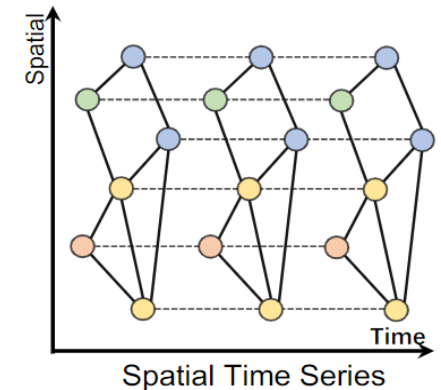
January 29, 2026

Space, Time, and Forecasting

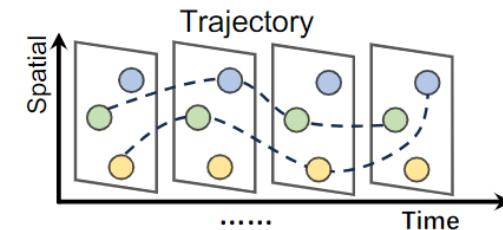
Time series is a set of observations, each one being recorded at a specific time, e.g., Annual GDP of UAE, iPhone Sales figure, etc.



A **spatial time series** is a type of data that combines spatial (cross-sectional) and temporal dimensions, capturing how measurements or observations vary across different locations over time, e.g., Air quality of Abu Dhabi.



Forecasting is estimating how the sequence of observations will continue into the future.



Time Series is omnipresent

- **Time series data** is a specialized form of data that plays a vital role in various fields, including economics, finance, climate science, healthcare, and many others.



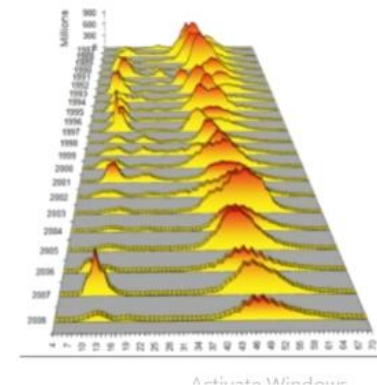
Shape of Time Series Data

- We usually think that the data is one-dimensional.
- It only consists of the time and the data associated with it (Temp.).

	New York City
1990-01-01	1
1990-01-02	5
1990-01-03	9
1990-01-04	13

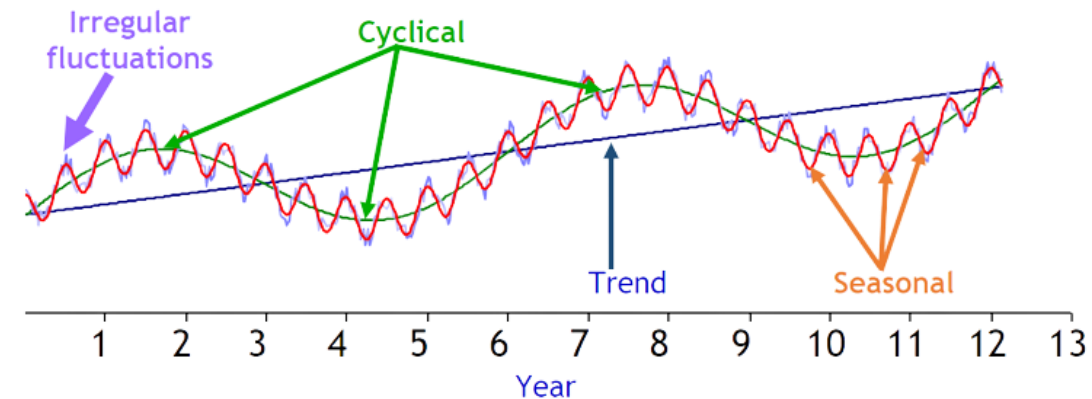
- But it can be Multidimensional.

	New York City	London	Tokyo	Paris
1990-01-01	1	2	3	4
1990-01-02	5	6	7	8
1990-01-03	9	10	11	12
1990-01-04	13	14	15	16



Time Series Components

- **Trend (T_t):** long-term increase or decrease in the time series.
- **Seasonal (S_t):** regular, repeating patterns at fixed intervals.
- **Cyclic (C_t):** Long-term fluctuation with no fixed periodicity.
- **Irregular (I_t):** Random, unpredictable fluctuations with no discernible pattern.
- **Decomposition :** $Y_t = f(T_t; S_t; C_t; I_t)$, where Y_t is data at period t .
- **Additive decomposition:** $Y_t = T_t + S_t + C_t + I_t$
- **Multiplicative decomposition:** $Y_t = T_t * S_t * C_t * I_t$



Forecasting

What people think I forecast?

When I go to any university, and I tell people that my job is time series forecasting and machine learning, usually one of two things happens:

- ...like, weather forecasting?

- Lots of domain knowledge and specialized models exist
- We leave it to meteorologists

- ...so, can you predict the stock market
and we all get rich?

- I'll tell you how, and we're all going to be rich!
- Try it on your own risk!

What I forecast?

- Epidemic time series (e.g., dengue, malaria, hepatitis, etc.)
- Sales forecasting in the supply chain, retail at pharmacy companies
- Forecasting in climate
 - Air quality
 - El Nino
 - Seismic events
- Key macroeconomic variables (inflation, unemployment, exchange rate, etc.)
- ...

Can these be forecasted?

1. daily electricity demand in 3 days' time
2. Google stock price tomorrow
3. Google stock price in 6 months' time
4. maximum temperature tomorrow
5. total sales of drugs in pharmacies next month

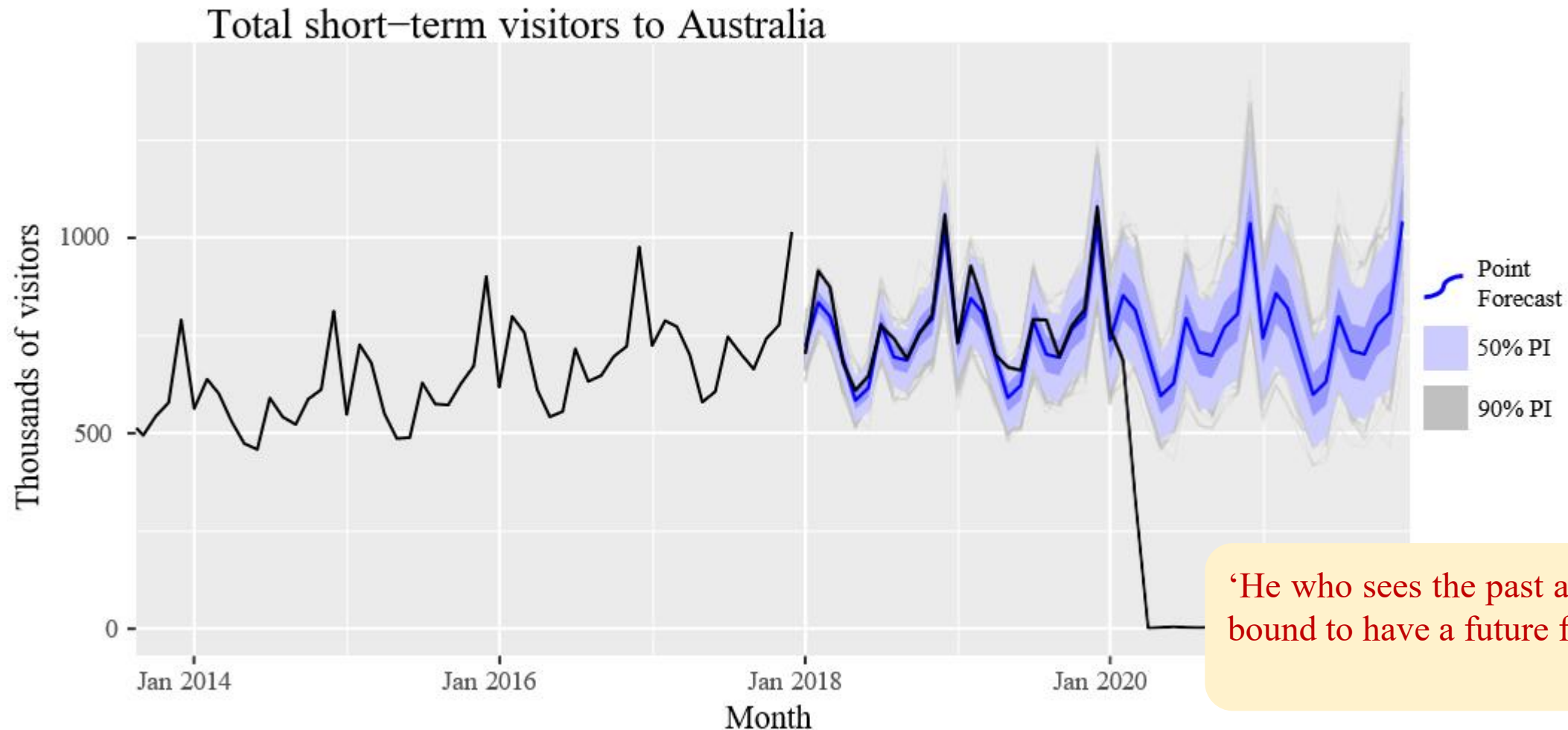


Something is easy to forecast if:

1. we have a good understanding of the factors that contribute to it
2. there is a lot of data available
3. the future is somewhat similar to the past
 - ID assumption: samples are identically distributed
4. the forecasts cannot affect the thing we are trying to forecast.
 - self-fulfilling prophecies (election polls)
 - controlled systems
 - Big bull effect in stock markets / bitcoin prices

Uncertainty and Forecasting

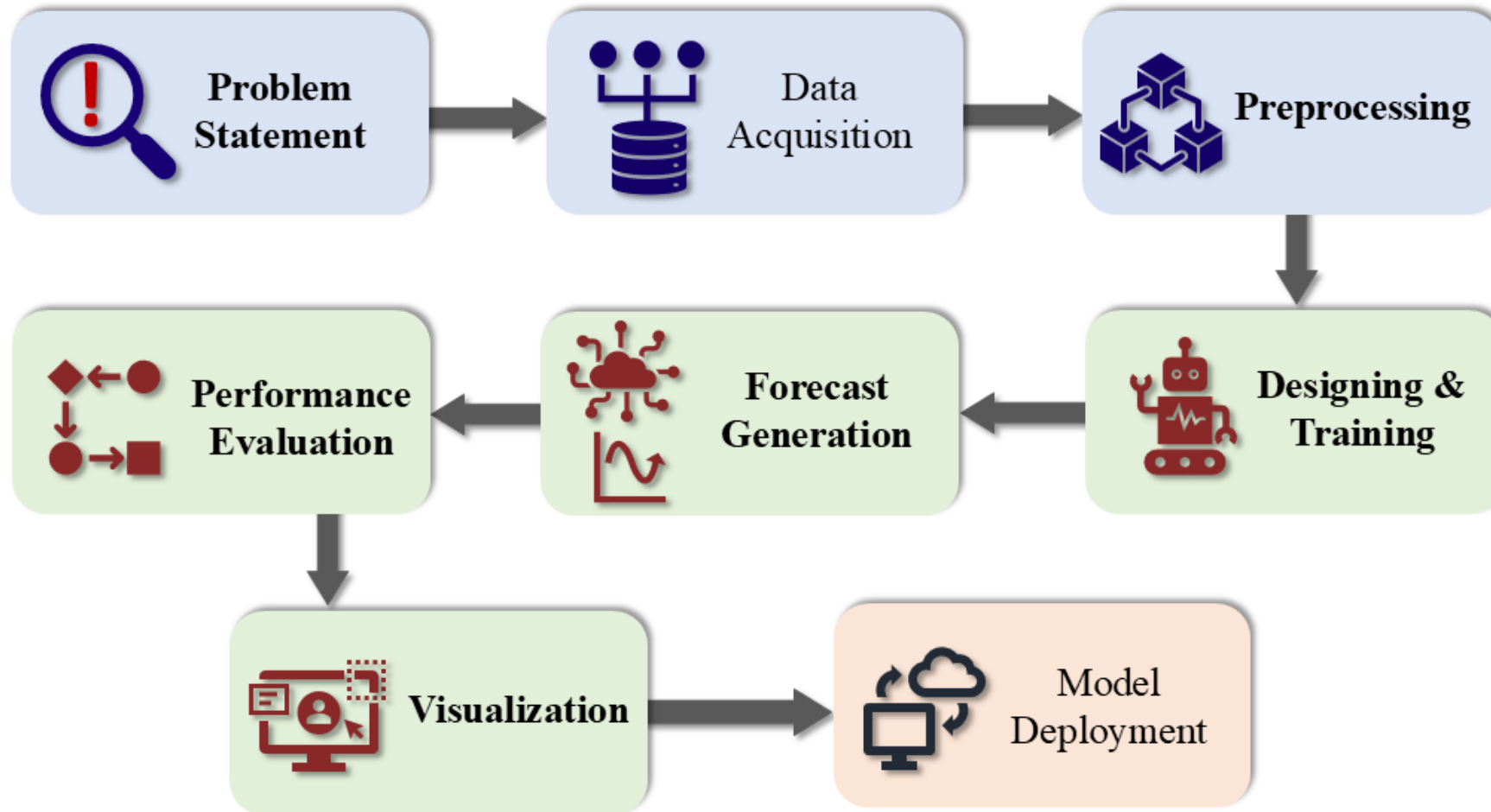
Forecasting is estimating how the sequence of observations will continue into the future.



‘He who sees the past as surprise-free is bound to have a future full of surprise.’

- Amos Tversky

Process of Forecasting



Forecasting workflow with main strata: data (blue rectangles), analytics (green rectangles), and deployment (orange rectangles)

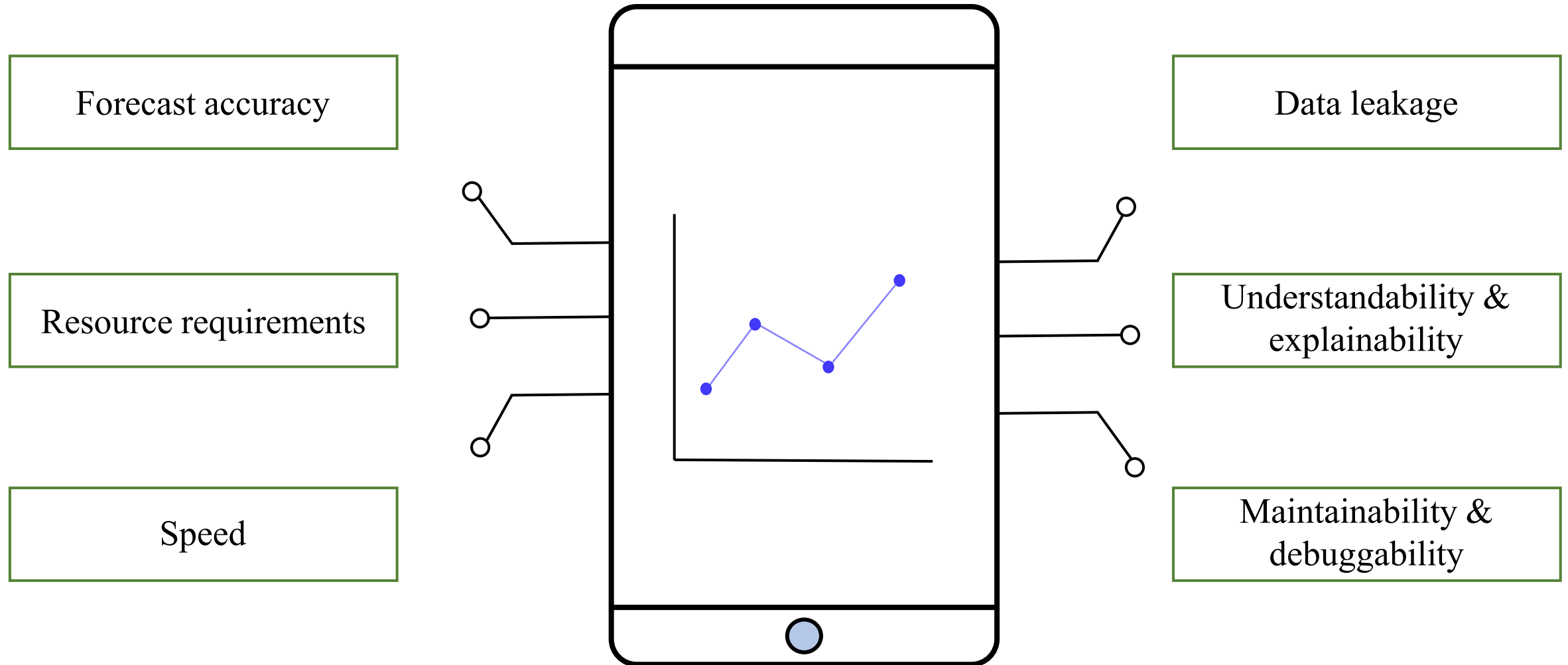
Time Series is omnipresent

- **Time series data** is a specialized form of data that plays a vital role in various fields, including economics, finance, climate science, healthcare, and many others.
- A **forecast** is a scientifically justified assertion about the possible states of an object in the future

Time Series is everywhere

- **Epidemiology:** Epidemic/Flu/Covid-19 cases observed over some time period.
- **Economics:** Stock prices, unemployment rates, inflation rates, etc.
- **Earth and Environmental Sciences:** Daily Sea Surface Temperature, Southern Oscillation Index, Seismic Waves, Air Quality Index, Global Warming, etc.
- **Astronomy:** Sunspot numbers, Luminosity of stars, etc.
- **Demography:** Population series, Birthrates, Mortality rates, etc.
- **Medical Science:** Blood pressure, Blood oxygen level, Sugar level, etc.
- **Business:** Product demand, Sales, Market share, etc.

What makes for a good forecasting model?



Forecast Performance Evaluation

Performance metrics such as mean absolute error (MAE), root mean square error (RMSE), and mean absolute percent error (MAPE) are used to evaluate the performances of different forecasting models for the unemployment rate data sets:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2};$$

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|;$$

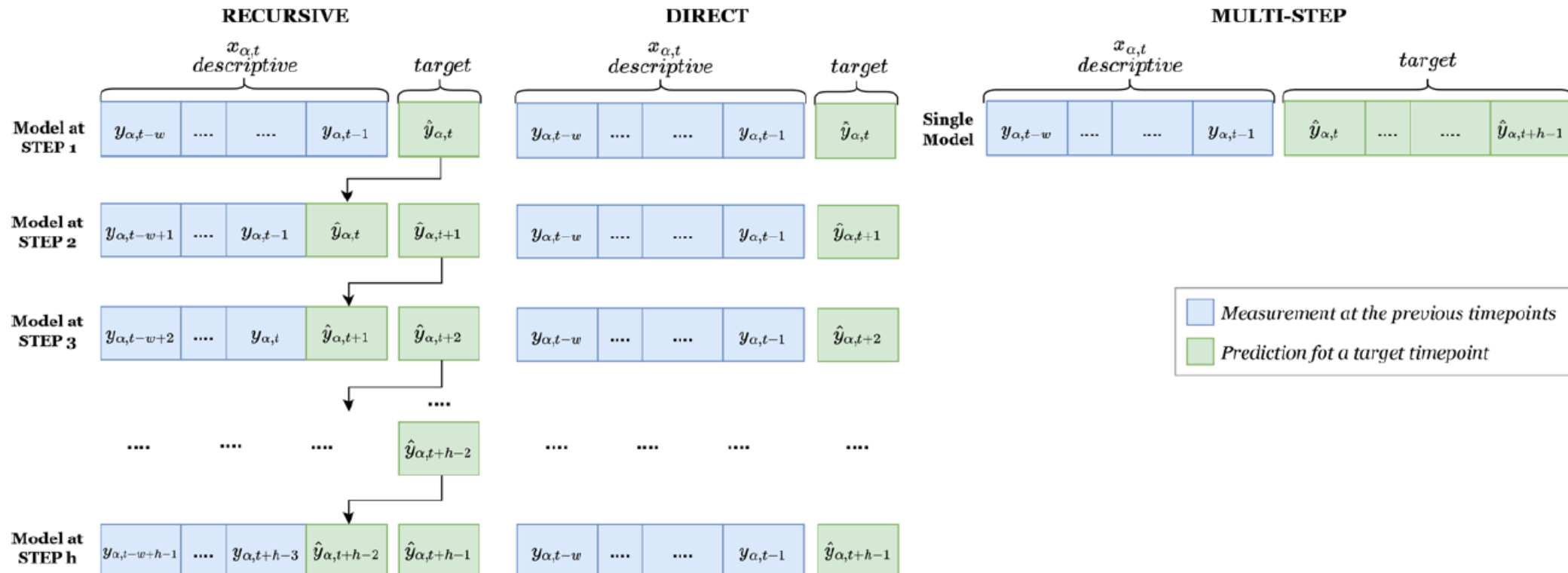
$$MAPE = \frac{1}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right|,$$

Where y_i is the actual output, $\hat{y}_i$ is the predicted output, and n denotes the number of data points.

By definition, the lower the value of these performance metrics, the better is the performance of the concerned forecasting model.

Types of Forecasting Tasks

1. One-Step Forecasting 2. Multi-Step Forecasting (Incremental Multi-step and Multi-output Multi step)



Classical Forecasting Methods

Average Method

- Here, the forecasts of all future values are equal to the average (or "mean") of the historical data. If we let the historical data be denoted by $y_1, \dots, y_T$, then we can write the forecasts as

$$\hat{y}_{T+h|T} = \bar{y} = (y_1 + \dots + y_T)/T.$$

- The notation $\hat{y}_{T+h|T}$ is a short-hand for the estimate of y_{T+h} based on the data $y_1, \dots, y_T$.

Naïve & Seasonal Naïve Method

- **Naïve Method:** All forecasts are set to be the value of the last observation. That is,

$$\hat{y}_{T+h|T} = y_T.$$

- This method works remarkably well for many economic and financial time series.
- **Seasonal Naïve Method:** For or highly seasonal data, forecasts are set equal to the most recent observation from the same season (e.g., the same month in the previous year):

$$\hat{y}_{T+h|T} = y_{T+h-m(k+1)},$$

where m = the seasonal period, and k is the integer part of $(h - 1)/m$ (i.e., the number of complete years in the forecast period prior to time $T + h$).

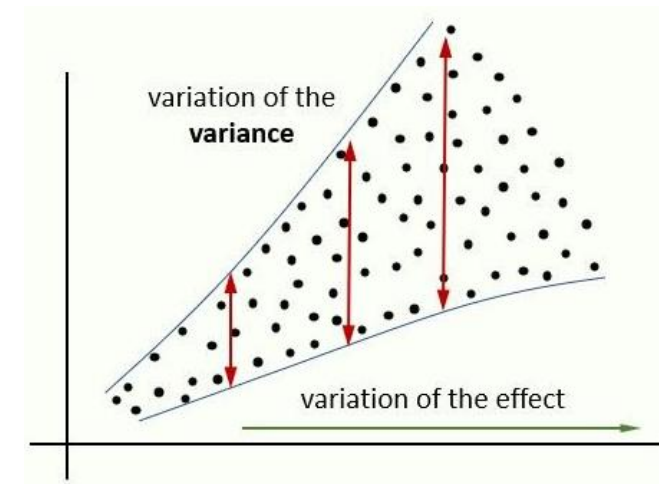
Auto-correlation Analysis

Auto Regression Analysis

- Regression analysis for time-ordered data is known as **Auto-Regression Analysis**
- **Time series data** are data collected on the same observational unit at multiple time period.



Example: Indian rate of price inflation



Heteroscedasticity

Modeling with Time Series Data

- Correlation over time
 - Serial correlation, also called autocorrelation
 - Calculating standard error
- To estimate dynamic causal effects
 - Under which dynamic effects can be estimated?
 - How to estimate?
- Forecasting model
 - Forecasting model build on regression model



Can we predict the trend (USD vs. INR) at a time, say 2025?

Auto-correlation coefficient

Just as [correlation](#) measures the extent of a linear relationship between two variables, [autocorrelation](#) measures the linear relationship between lagged values of a time series.

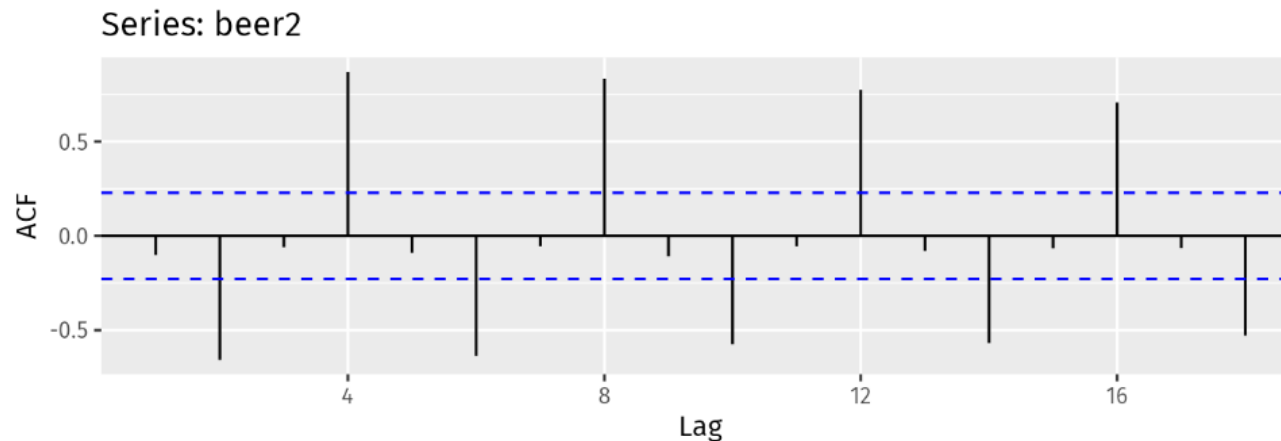
- There are several autocorrelation coefficients, corresponding to each panel in the lag plot.
- For example, let $[y_1, y_2, \dots, y_{T-1}, y_T]$ denote T observations on the time series random variable Y_t , then r_1 measures the relationship between y_t and y_{t-1} , r_2 measures the relationship between y_t and y_{t-2} , and so on.

The value of r_k can be written as

$$r_k = \frac{\sum_{t=k+1}^T (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^T (y_t - \bar{y})^2}$$

where T is the length of the time series. The autocorrelation coefficients make up the autocorrelation function or ACF.

The autocorrelation coefficients are plotted to show the [autocorrelation function](#) or ACF of quarterly beer production in Australia. The plot is also known as a [correlogram](#).

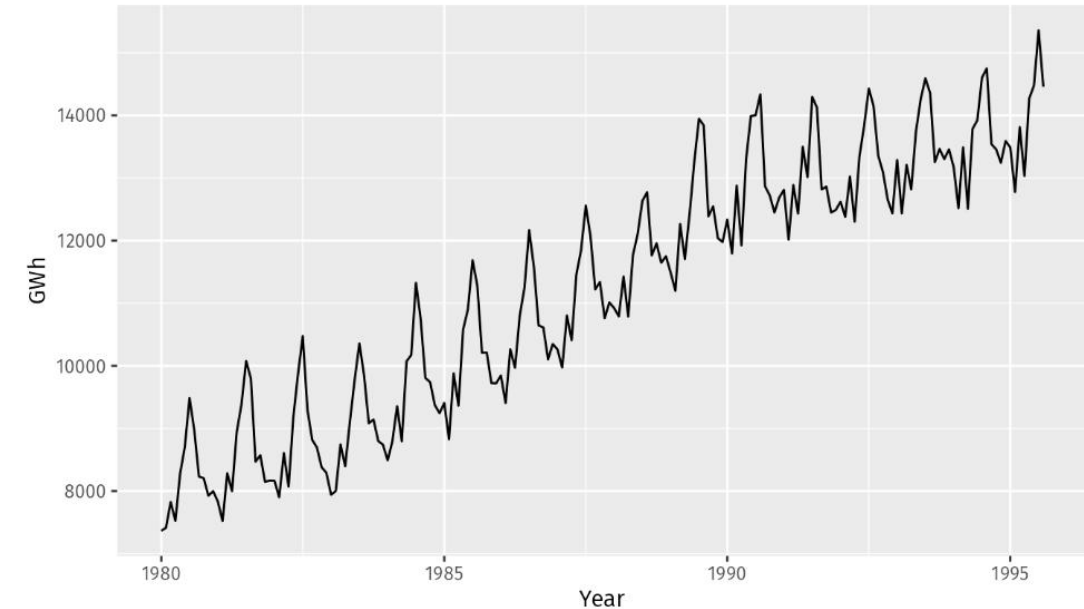


In this graph:

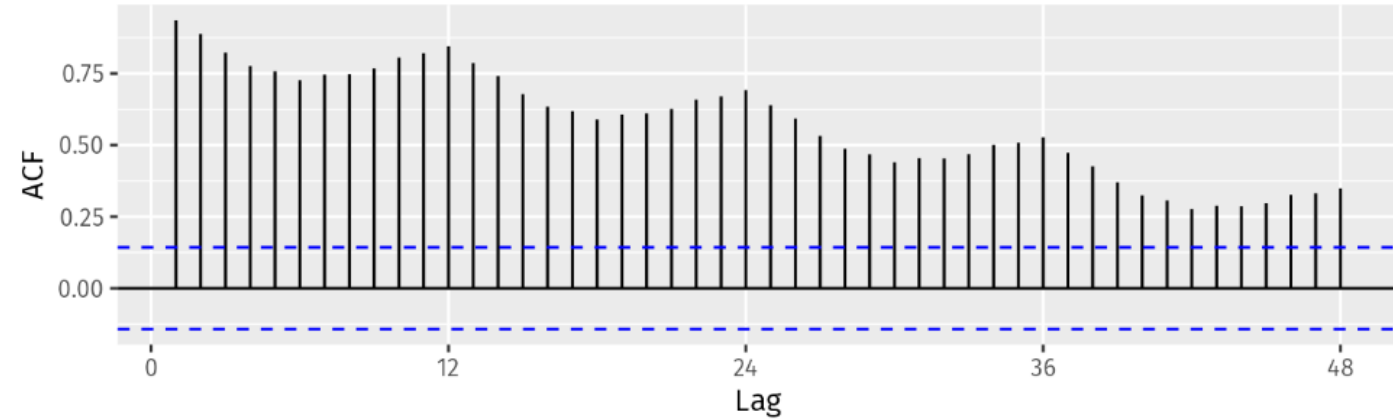
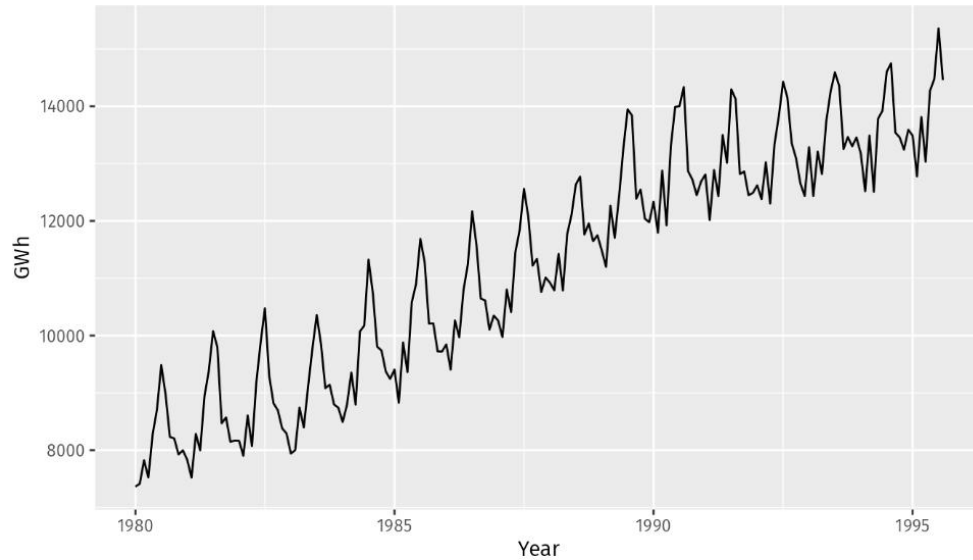
- r_4 is higher than for the other lags. This is due to the seasonal pattern in the data: the peaks tend to be four quarters apart and the troughs tend to be four quarters apart.
- r_2 is more negative than for the other lags because troughs tend to be two quarters behind peaks.
- The dashed blue lines indicate whether the correlations are significantly different from zero.

Example: Autocorrelation

- When data have a **trend**, the autocorrelations for small lags tend to be large and positive because observations nearby in time are also nearby in size. So the ACF of trended time series tend to have positive values that slowly decrease as the lags increase.
- When data are **seasonal**, the autocorrelations will be larger for the seasonal lags (at multiples of the seasonal frequency) than for other lags.
- When data are both **trended and seasonal**, you see a combination of these effects.
- The monthly Australian electricity demand series plotted in Figure shows both **trend and seasonality**.



Example: Autocorrelation



The slow decrease in the ACF as the lags increase is due to the [trend](#), while the “scalloped” shape is due to the [seasonality](#).

Auto-Regression Model

Auto-Regression Model

An **autoregressive model (also called AR model)** is used to model a future behavior for a time-ordered data, using data from past behaviors.

- Essentially, it is a linear regression analysis of a dependent variable using one or more variables(s) in a given time-series data: $Y_t = f(Y_{t-1}, Y_{t-2}, \dots, Y_{t-p})$
- A natural starting point for forecasting model is to use past values of Y , that is, $Y_{t-1}, Y_{t-2}, \dots$ to predict Y_t .
- An auto-regression is a regression model in which Y_t is regressed against its own lagged values.
- The number of lags used as regressors is called the **order of auto-regression**.
 - In first order auto-regression (denoted as $AR(1)$), Y_t is regressed against Y_{t-1}
 - In p^{th} order auto-regression (denoted as $AR(p)$), Y_t is regressed against, $Y_{t-1}, Y_{t-2}, \dots, Y_{t-p}$

Auto-Regression Model for Forecasting

An **AutoRegressive model of order 1** (AR(1)) says: Today's value = $\phi \times$ Yesterday's value + shock

More formally:

$$Y_t = \phi Y_{t-1} + \epsilon_t$$

Where:

- Y_t : value of the variable at time t (e.g., stock return, inflation, sales).
- ϕ : a parameter that tells us how much of yesterday's value spills into today.
- ϵ_t : a random shock (news, surprises, etc.) $\epsilon_t \sim \text{iid } N(0, \sigma^2)$ is white noise.

Example:

Suppose a stock's daily return depends slightly on the previous day:

$$R_t = 0.4R_{t-1} + \epsilon_t$$

If yesterday's return was 2%, then **today's expected return** is:

$$\mathbb{E}[R_t] = 0.4 \times 2\% = 0.8\%$$

p^{th} order Auto-regression Model

Formula: p^{th} Order Auto-regression Model

In general, the p^{th} order auto-regression model is defined as

$$Y_t = \beta_0 + \sum_{i=1}^p \beta_i Y_{t-i} + \varepsilon_t$$

where, $\beta_0, \beta_1, \dots, \beta_p$ is called auto-regression coefficients and ε_t is the noise term or residue and in practice it is assumed to Gaussian white noise

Computing AR Coefficients

Method 1: Ordinary Least Squares (OLS)

$$Y_t = \phi Y_{t-1} + \epsilon_t, \quad t = 2, \dots, T$$

Step-by-step derivation

1. Define the sum of squared residuals (SSR):

$$S(\phi) = \sum_{t=2}^T (Y_t - \phi Y_{t-1})^2$$

2. Differentiate $S(\phi)$ with respect to ϕ :

$$\frac{dS(\phi)}{d\phi} = \sum_{t=2}^T 2(Y_t - \phi Y_{t-1})(-Y_{t-1}) = -2 \sum_{t=2}^T Y_{t-1}(Y_t - \phi Y_{t-1})$$

Computing AR Coefficients

Method 1: Ordinary Least Squares (OLS)

3. Set the derivative to zero for minimization:

$$\sum_{t=2}^T Y_{t-1}(Y_t - \phi Y_{t-1}) = 0$$

4. Solve for ϕ :

$$\sum_{t=2}^T Y_{t-1}Y_t = \phi \sum_{t=2}^T Y_{t-1}^2 \Rightarrow \hat{\phi} = \frac{\sum_{t=2}^T Y_{t-1}Y_t}{\sum_{t=2}^T Y_{t-1}^2}$$

5. Estimate residual variance:

$$\hat{\sigma}^2 = \frac{1}{T-1} \sum_{t=2}^T (Y_t - \hat{\phi}Y_{t-1})^2$$

AR Model by Hand

Here is a small dataset of 5 time points used for manually fitting the AR(1) model.

	Time	Y
1	1	1.2
2	2	1.8
3	3	2.4
4	4	2.1
5	5	1.9

AR Model by Hand

1. "Step-by-step AR(1) Estimation (Small Data)" – This table shows:

- Y_{t-1} (lagged values)
- Y_t (current values)
- Predicted Y_t using $\hat{\phi}Y_{t-1}$
- Residuals = $Y_t - \hat{\phi}Y_{t-1}$

2. "Estimated Parameters (Small Data)" – This table displays:

- Estimated $\hat{\phi} \approx 1.044$
- Estimated $\hat{\sigma} \approx 0.453$

Y_{t-1}	Y_t	Predicted	Residual
1.2	1.8	1.25333	0.54667
1.8	2.4	1.88	0.52
2.4	2.1	2.50667	-0.40667
2.1	1.9	2.19333	-0.29333

These values are derived manually (without libraries) using the formulas:

- $\hat{\phi} = \frac{\sum Y_{t-1}Y_t}{\sum Y_{t-1}^2}$
- $\hat{\sigma}^2 = \frac{1}{T-1} \sum (Y_t - \hat{\phi}Y_{t-1})^2$

Applications in Business and Finance

In Finance:

- **Interest rates** tend to follow an AR(1) pattern — central banks move gradually.
- **Volatility forecasting** uses AR-type models in GARCH models.
- **Stock returns** may have slight autocorrelation due to investor herding.

In Business:

- **Monthly sales**, if not seasonally adjusted, often follow AR(1)-like behavior.
- **Customer demand** today may depend on yesterday's trend.

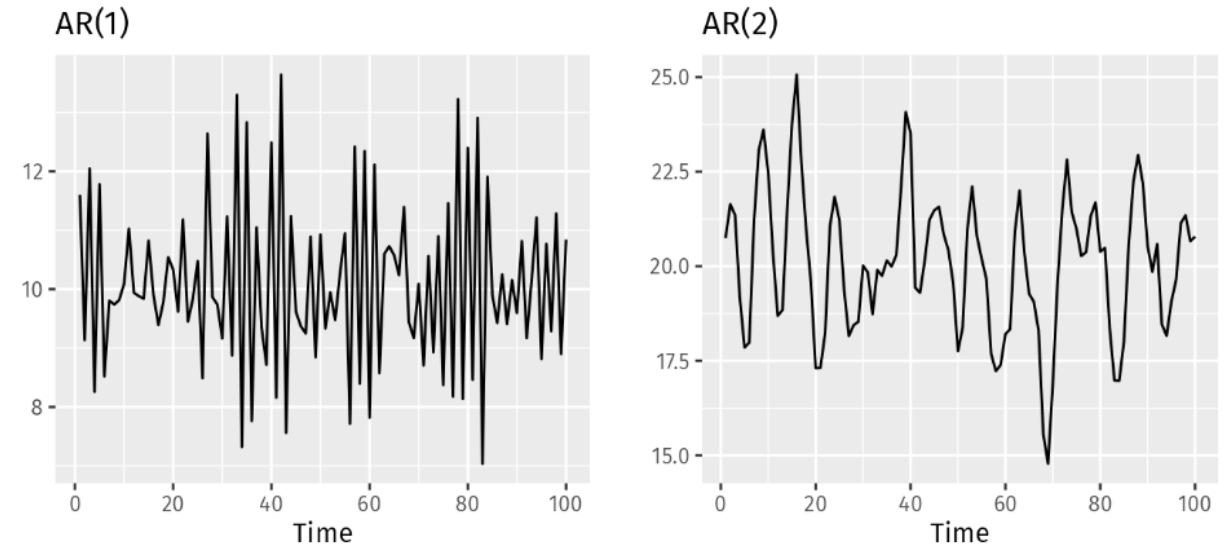


Figure 8.5: Two examples of data from autoregressive models with different parameters. Left: AR(1) with $y_t = 18 - 0.8y_{t-1} + \varepsilon_t$. Right: AR(2) with $y_t = 8 + 1.3y_{t-1} - 0.7y_{t-2} + \varepsilon_t$. In both cases, ε_t is normally distributed white noise with mean zero and variance one.

Applications in Business and Finance

Application	What AR Model Helps With
Inventory management	Forecast next week's demand
Revenue forecasting	Predict next quarter's sales
Risk management	Model interest rate paths
Trading strategies	Test autocorrelation of returns

Key Takeaways:

- AR models use past information — simple yet powerful.
- AR(1) means "today is a shadow of yesterday, plus some new noise."
- Widely used in finance (returns, rates) and business (sales, demand).
- Easy to implement, interpret, and extend.

ARIMA Model

ARIMA

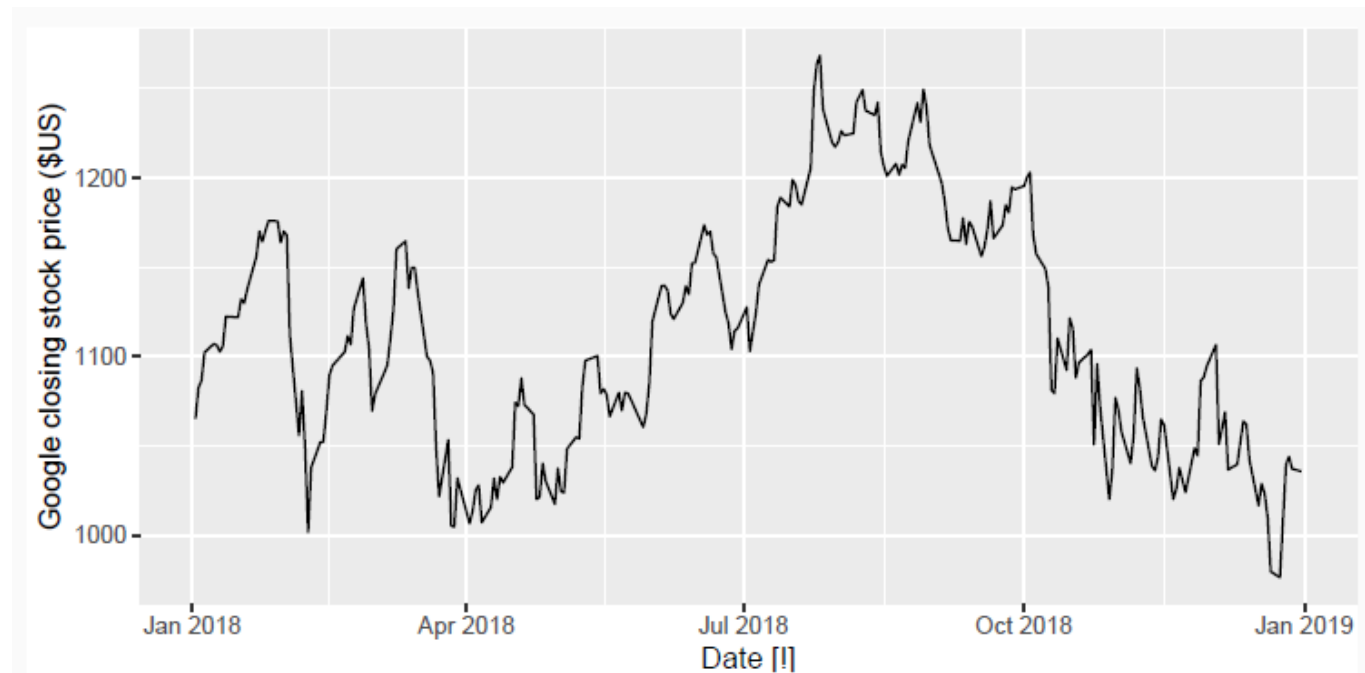
- **AR:** autoregressive (lagged observations as inputs)
- **I:** integrated (differencing to make series stationary)
- **MA:** moving average (lagged errors as inputs)

An ARIMA model is rarely interpretable in terms of visible data structures like trend and seasonality. But it can capture a huge range of time series patterns.

Stationary

Definition: If $\{y_t\}$ is a stationary time series, then for all s , the distribution of $(y_t, \dots, y_{t+s})$ does not depend on t .

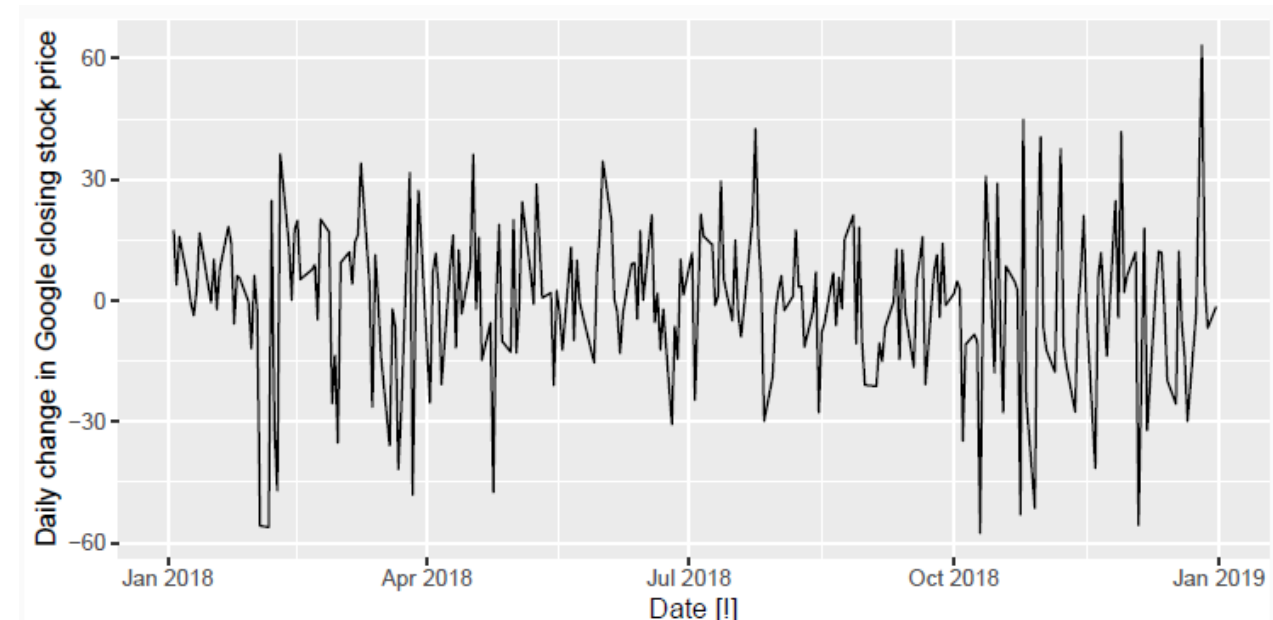
- A stationary series is:
- roughly horizontal
- constant variance
- no patterns predictable in the long-term



Daily close price of Google Stock during Jan – Dec 2018.

Differencing

- Differencing helps to **stabilize the mean**.
- The differenced series is the change between each observation in the original series.
- Occasionally the differenced data will not appear stationary and it may be necessary to difference the data a second time.
- In practice, it is almost never necessary to go beyond second-order differences.



Daily change in close price of Google Stock during Jan – Dec 2018.

ARIMA models

Autoregressive Moving Average models:

$$y_t = c + \phi_1 y_{t-1} + \cdots + \phi_p y_{t-p} + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q} + \varepsilon_t$$

- Predictors include both lagged values of y_t and lagged errors.
- Combine ARMA model with differencing.
- d-differenced series follows an ARMA model.
- Need to choose p, d, q and whether or not to include c .

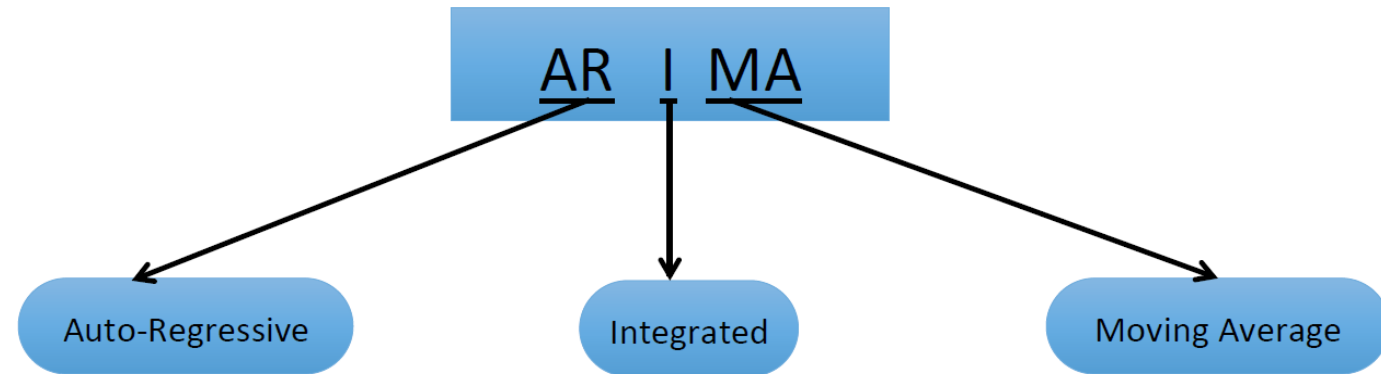
ARIMA models

ARIMA(p, d, q) model

AR: p = order of the autoregressive part

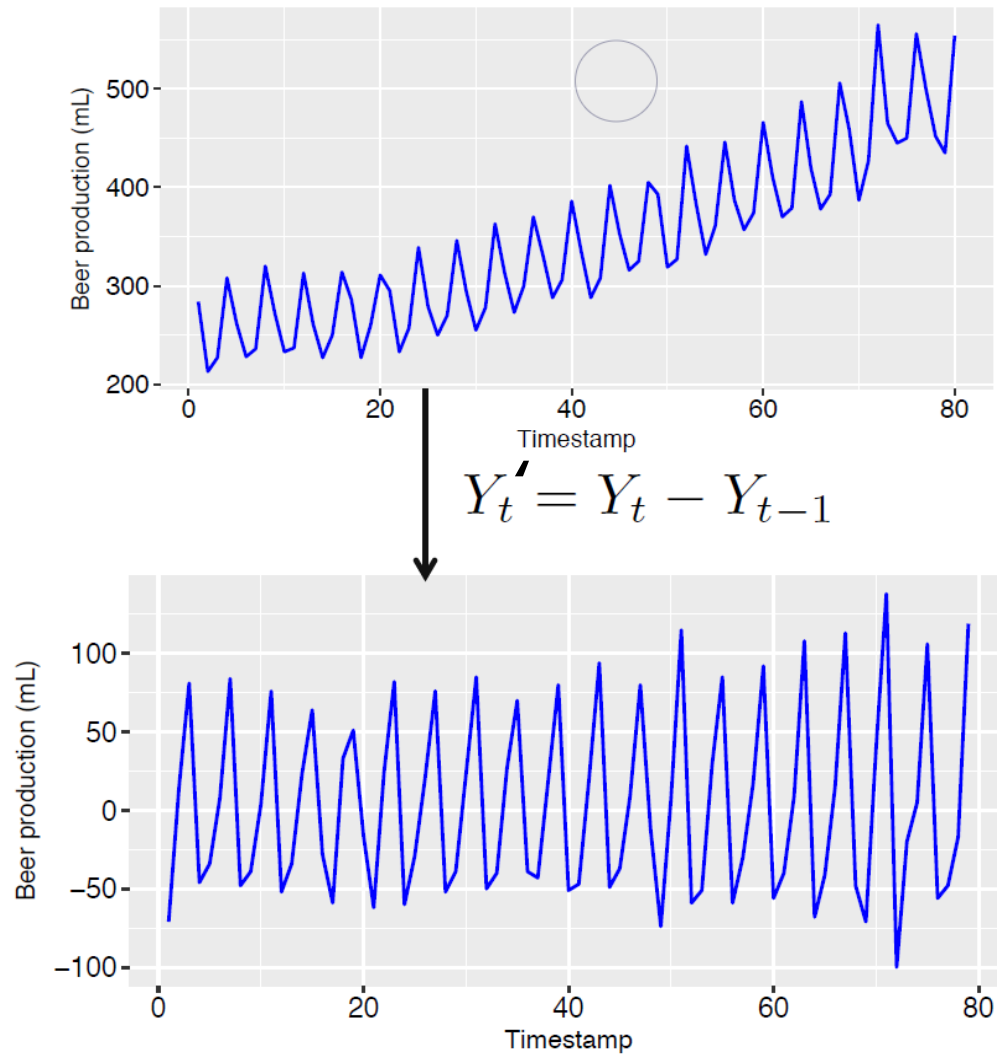
I: d = degree of first differencing involved

MA: q = order of the moving average part.



- White noise model: ARIMA(0,0,0)
- Random walk: ARIMA(0,1,0) with no constant
- Random walk with drift: ARIMA(0,1,0) with const.
- AR(p): ARIMA($p,0,0$)
- MA(q): ARIMA(0,0, q)

How to select d ?



Differencing order (d):
Number of times differencing is done

Selecting p and q manually

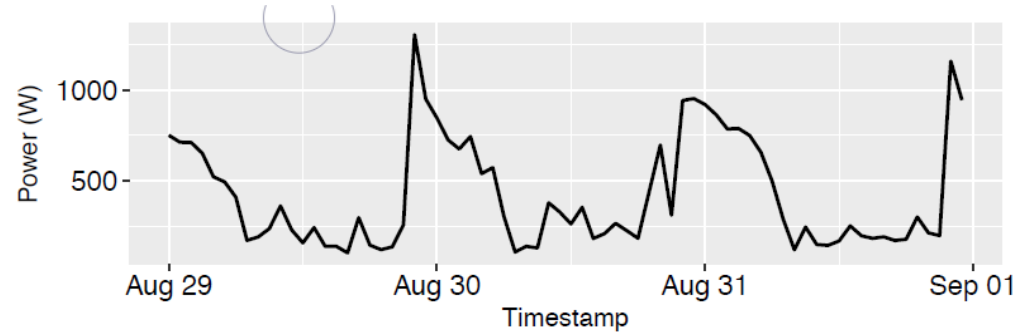
1. Auto-Correlation Function (ACF) Plot:

- Correlation coefficients of time-series at different lags
- Defines q order of MA model

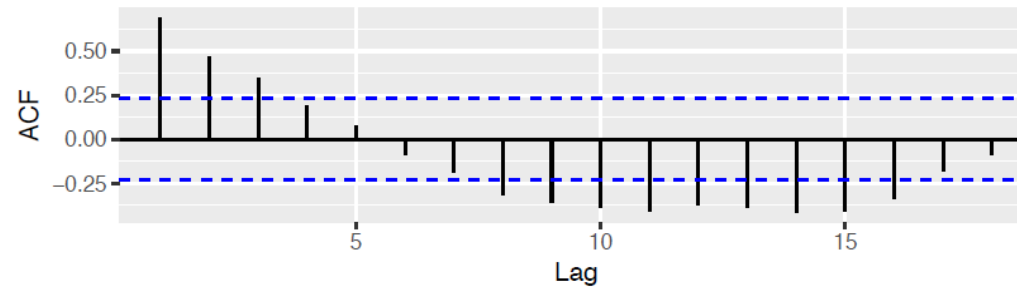
2. Partial Auto-correlation Function (PACF) Plot:

- Partial correlation coefficients of time series at different lags
- Defines p order of AR model

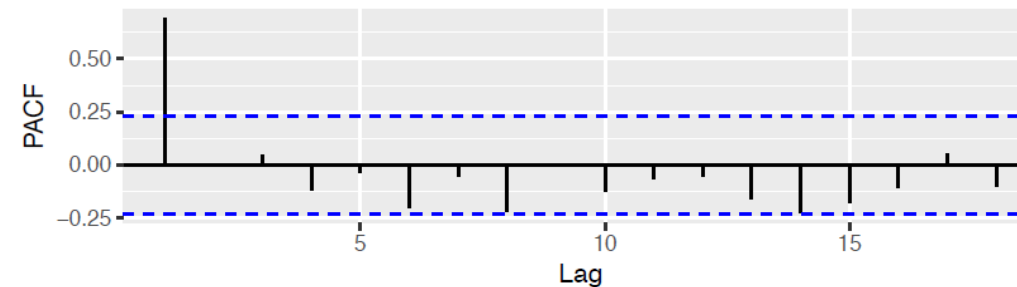
ACF / PACF Plots : Example



Data



ACF Plot

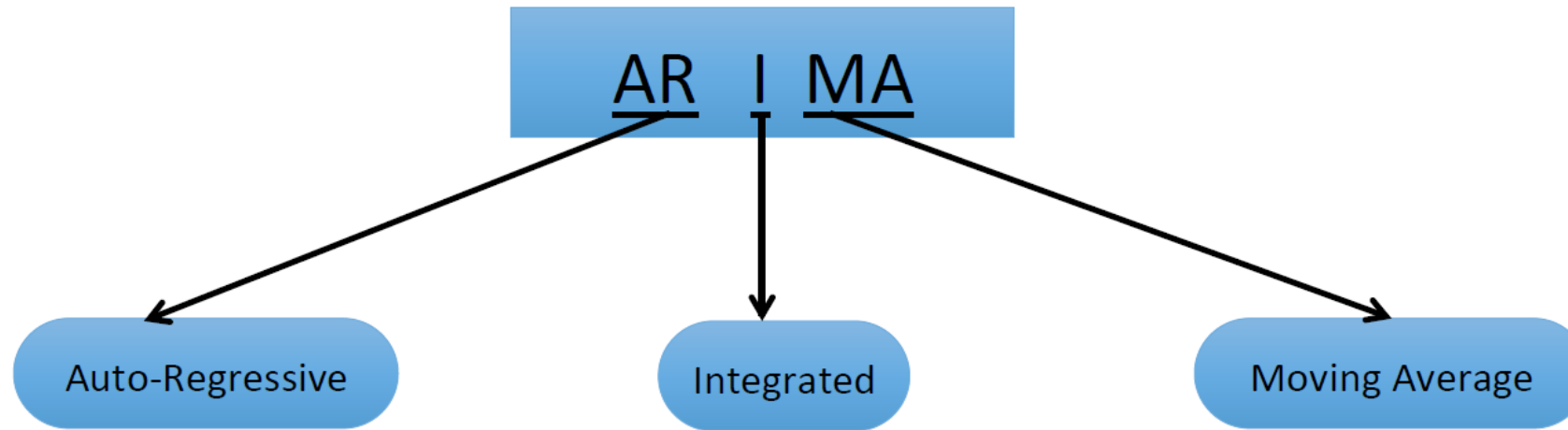


PACF Plot

Understanding ARIMA models

- If $c = 0$ and $d = 0$, the long-term forecasts will go to zero.
- If $c = 0$ and $d = 1$, the long-term forecasts will go to a non-zero constant.
- If $c = 0$ and $d = 2$, the long-term forecasts will follow a straight line.
- If $c = 0$ and $d = 0$, the long-term forecasts will go to the mean of the data.
- If $c = 0$ and $d = 1$, the long-term forecasts will follow a straight line.
- If $c = 0$ and $d = 2$, the long-term forecasts will follow a quadratic trend.

ARIMA models



$ARIMA(p, d, q)$ model

AR: p = order of the autoregressive part

I: d = degree of first differencing involved

MA: q = order of the moving average part.

How does ARIMA select parameters?

- Select no. differences d via differencing tests, such as Augmented Dickey Fuller (ADF) test.
- Select p, q and inclusion of c by minimizing [Corrected Akaike's Information Criterion](#) (AICc).
- Use stepwise search to traverse model space.

$$\text{AICc} = -2\log(L) + 2(p + q + k + 1) \left[1 + \frac{(p + q + k + 2)}{T - p - q - k - 2} \right]$$

where L is the maximised likelihood fitted to the differenced data, $k = 1$ if $c \neq 0$ and $k = 0$ otherwise.

Note: Can't compare AICc for different values of d .

How does ARIMA work?

Step 1: Select current model (with smallest AICc) from:

ARIMA(2, d , 2)

ARIMA(0, d , 0)

ARIMA(1, d , 0)

ARIMA(0, d , 1)

Step 2: Consider variations of current model:

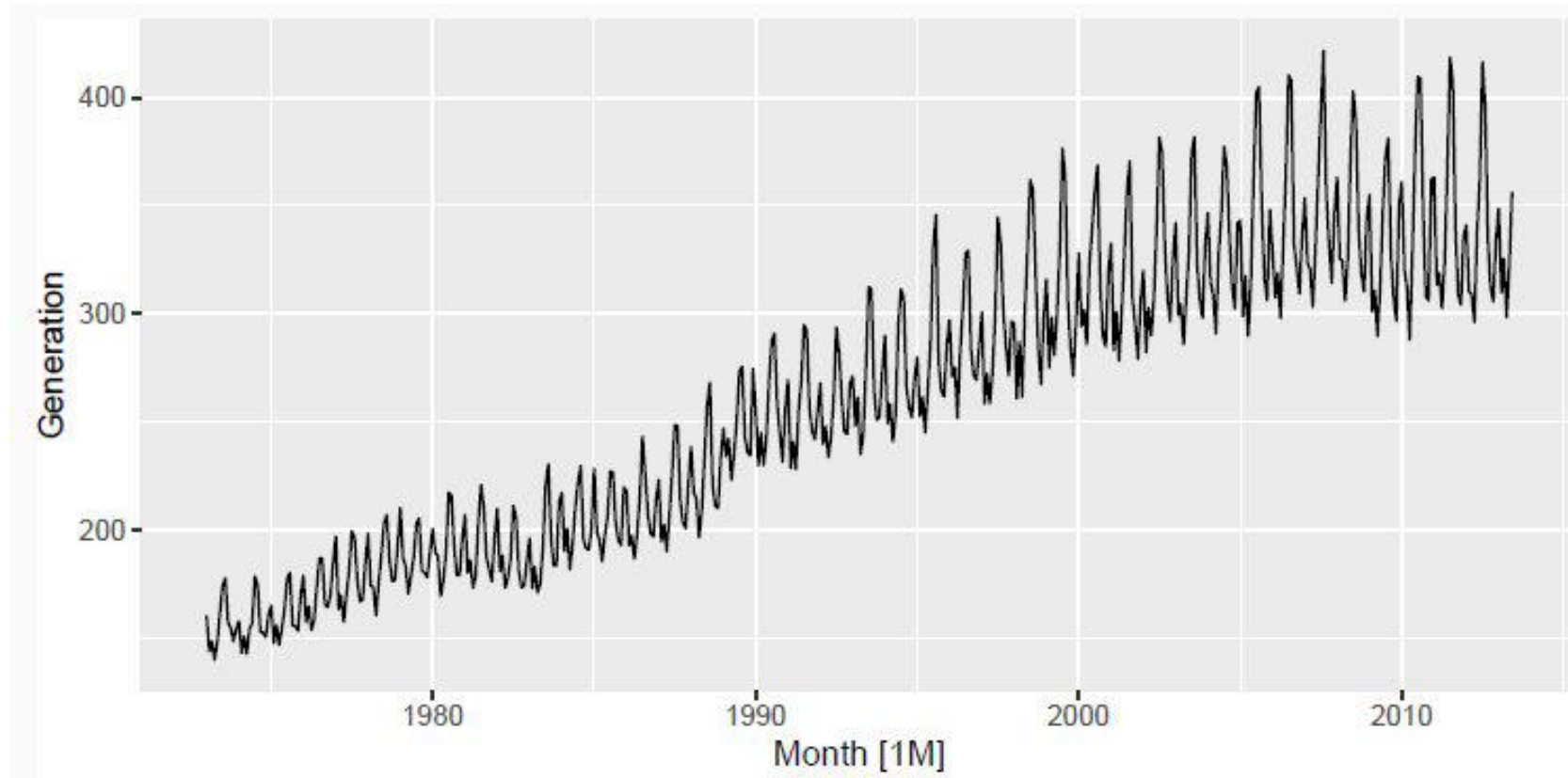
- vary one of p, q , from current model by ± 1 ;
- p, q both vary from current model by ± 1 ;
- Include/exclude c from current model.

Model with lowest AICc becomes current model.

Repeat Step 2 until no lower AICc can be found.

Seasonal ARIMA

- Many real-world time series, like temperature, sales, rainfall, electricity consumption, exhibit seasonal patterns that repeat at regular intervals. ARIMA alone cannot handle such seasonal effects effectively.



Monthly US electricity generation data from 1972 to 2013

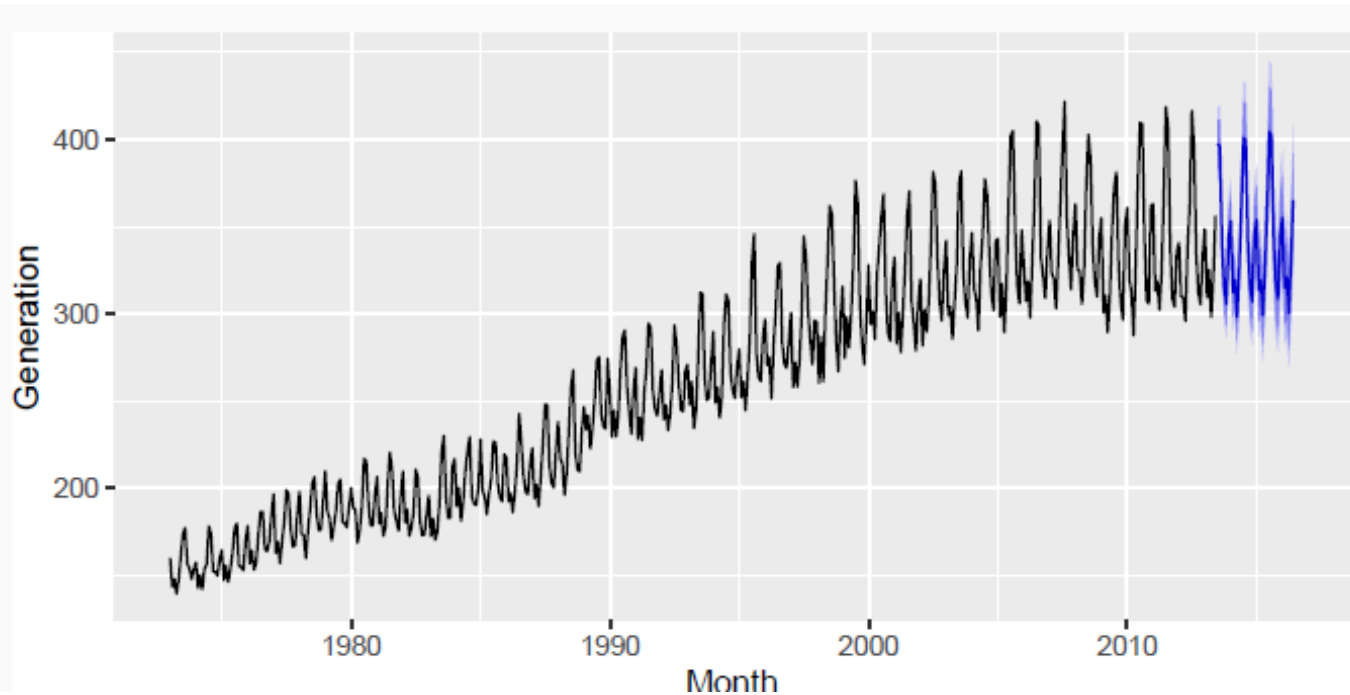
Seasonal ARIMA

ARIMA	(p, d, q)	$(P, D, Q)_m$
	↑	↑
	Non-seasonal part of the model	Seasonal part of of the model

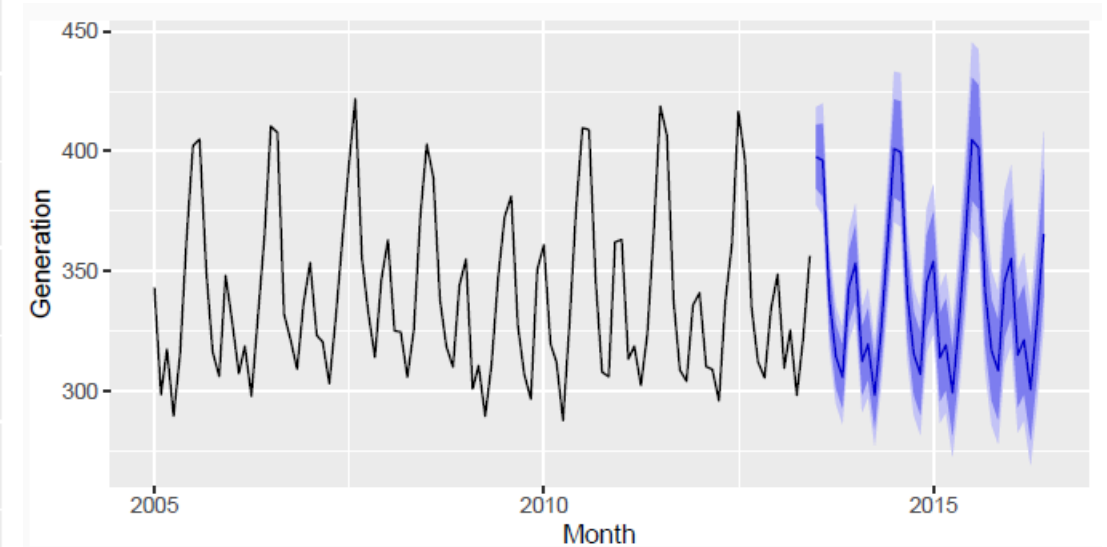
- m = number of observations per year.
- d first differences, D seasonal differences
- p AR lags, q MA lags
- P seasonal AR lags, Q seasonal MA lags

Seasonal and non-seasonal terms combine
multiplicatively

Seasonal ARIMA Model Forecast



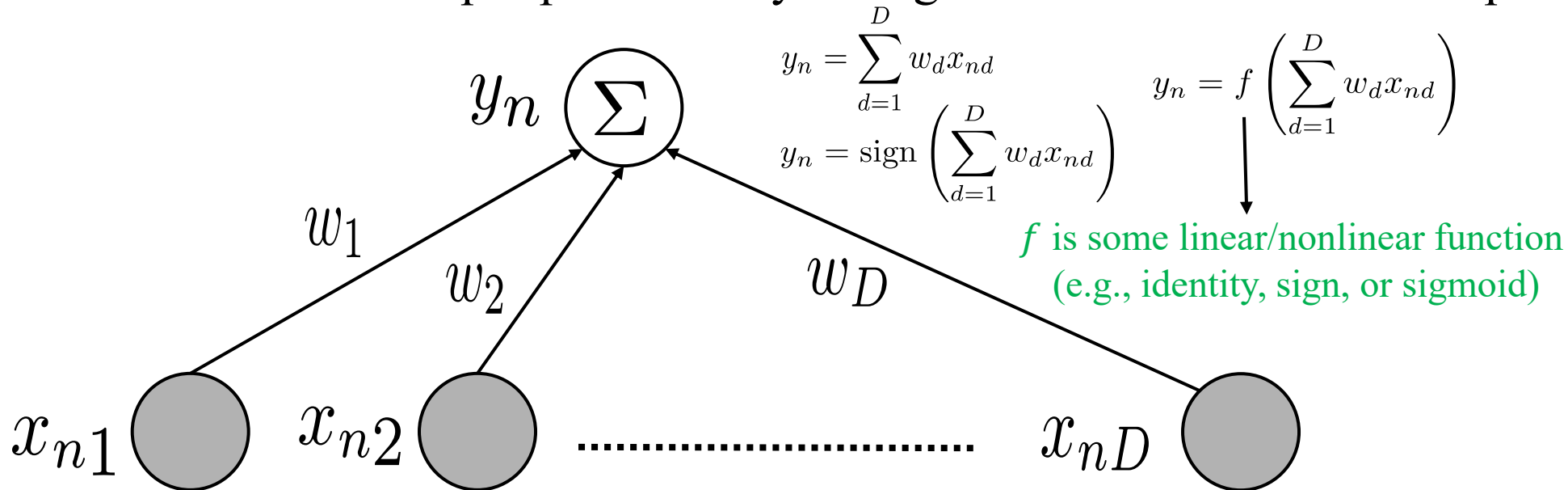
Monthly US electricity generation forecast for 2014 - 2016



Forecast horizon zoomed in

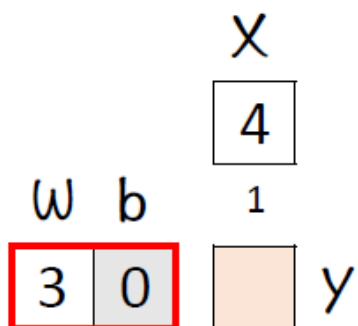
Limitation of Linear Models

- Linear models: Output produced by taking a linear combination of input features

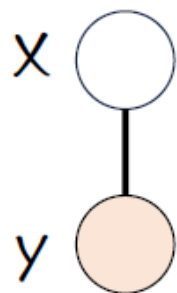


- A basic unit of the form $y = f(\mathbf{w}^\top \mathbf{x})$ is known as the “Perceptron” (not to be confused with the Perceptron “algorithm”, which learns a linear classification model)
- This can’t however learn nonlinear functions or nonlinear decision boundaries

Exercise: Linear Layer

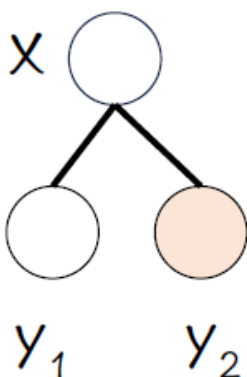
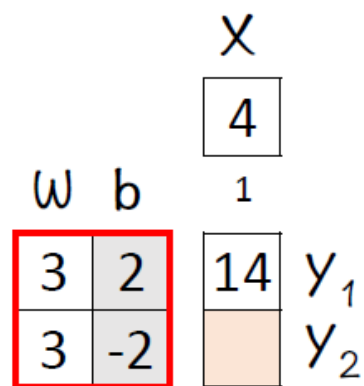


$$y = [w|b] \cdot [X|1] = wX + b$$



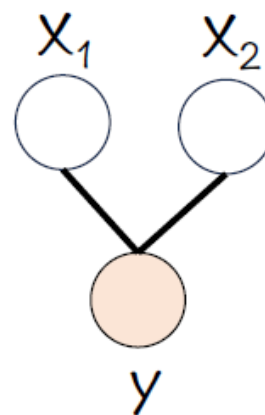
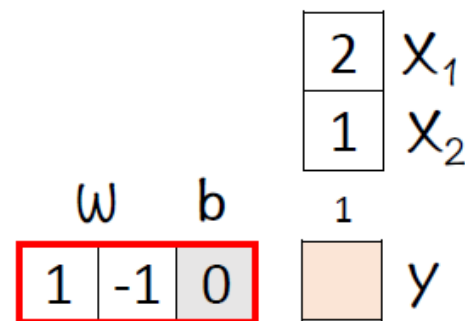
Exercise 1

12



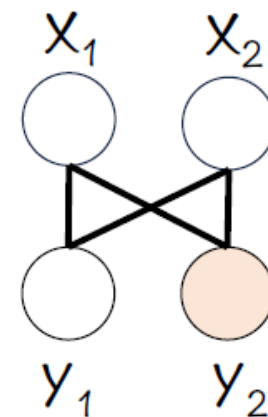
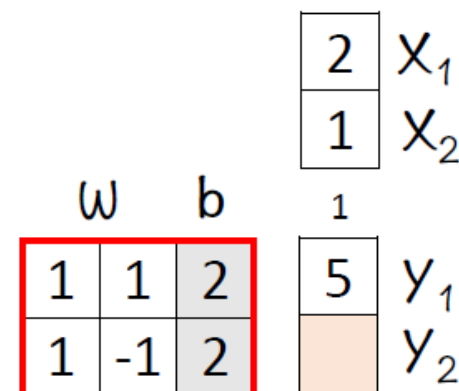
Exercise 2

10



Exercise 3

1

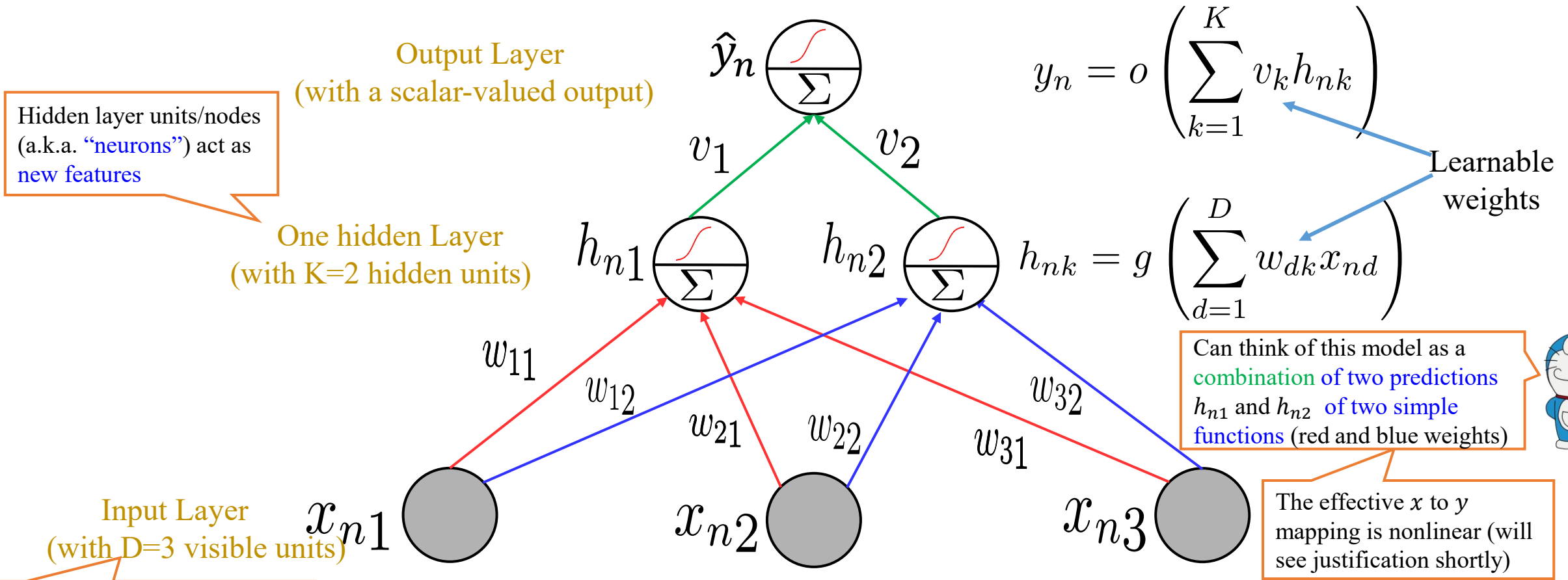


Exercise 4

3

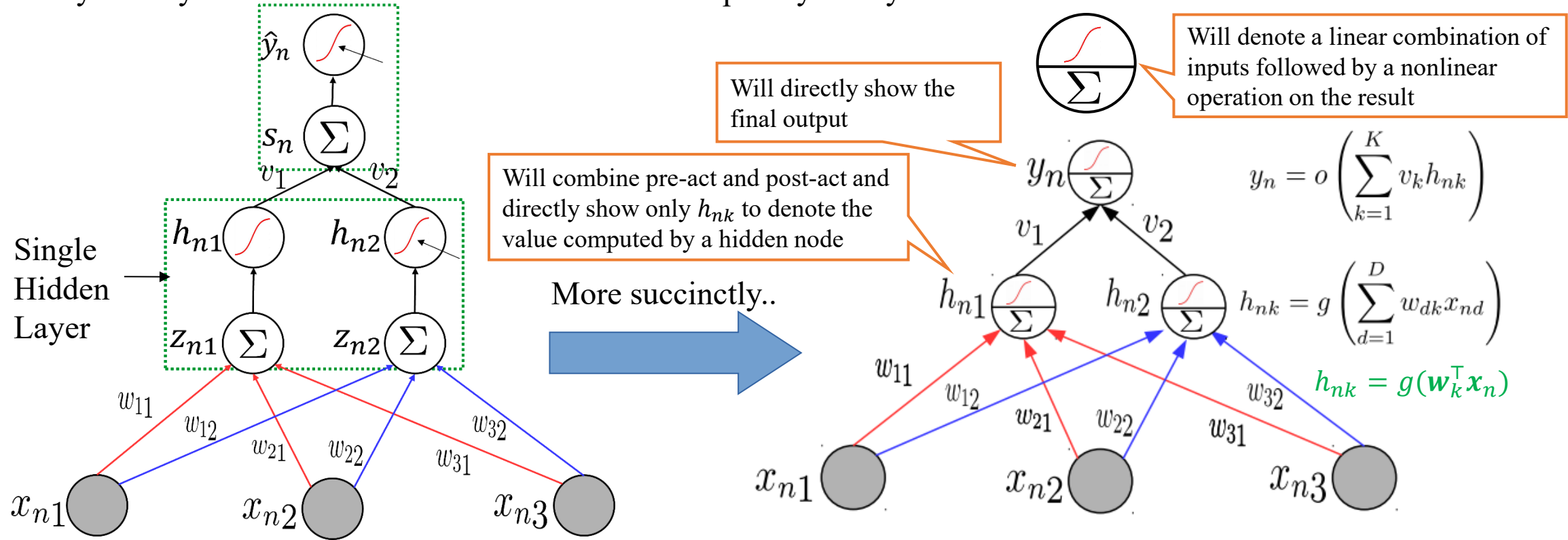
Neural Networks: Multi-layer Perceptron (MLP)

- An MLP consists of an **input layer**, an **output layer**, and **one or more hidden layers**



Neural Nets: A Compact Illustration

- Note: Hidden layer pre-act z_{nk} and post-act h_{nk} will be shown together for brevity
- Different layers may use different non-linear activations. Output layer may have none.

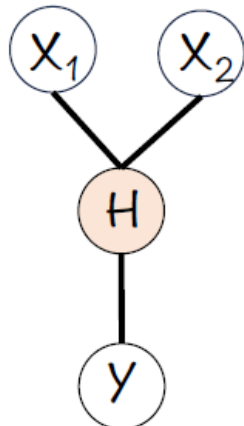


- Denoting $\mathbf{W} = [\mathbf{w}_1, \mathbf{w}_2, \dots, \mathbf{w}_K]$, $\mathbf{w}_k \in \mathbb{R}^D$, $\mathbf{h}_n = g(\mathbf{W}^\top \mathbf{x}_n) \in \mathbb{R}^K$ ($K = 2, D = 3$ above).
- **Note:** g applied elementwise on pre-activation vector $\mathbf{z}_n = \mathbf{W}^\top \mathbf{x}_n$

Exercise: Hidden Layer

$$\begin{matrix} w & b \\ \boxed{1} & \boxed{2} & \boxed{0} \end{matrix}$$

$$\begin{matrix} 1 \\ \boxed{1} \\ -1 \end{matrix} \begin{matrix} X_1 \\ X_2 \\ 1 \end{matrix} \xrightarrow{\text{ReLU}} \begin{matrix} \boxed{2} \\ -1 \end{matrix} \approx \begin{matrix} \boxed{1} \\ -1 \end{matrix} \xrightarrow{\text{ReLU}} \boxed{0} \quad y$$



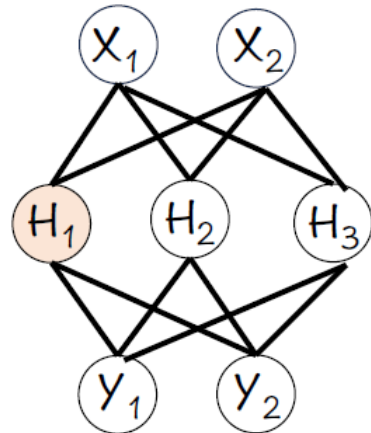
Exercise 1

$$\begin{matrix} \boxed{-1} & \boxed{0} \end{matrix}$$

$$\begin{matrix} 3 \\ 1 \end{matrix} \begin{matrix} X_1 \\ X_2 \end{matrix}$$

$$\begin{matrix} \boxed{0} & \boxed{2} & \boxed{1} \\ \boxed{1} & \boxed{1} & \boxed{-1} \\ \boxed{1} & \boxed{0} & \boxed{-1} \end{matrix} \xrightarrow{\text{ReLU}} \begin{matrix} \boxed{3} \\ 2 \end{matrix} \approx \begin{matrix} \boxed{3} \\ 2 \end{matrix} \begin{matrix} H_1 \\ H_2 \\ H_3 \end{matrix}$$

$$\begin{matrix} \boxed{1} & \boxed{1} & \boxed{0} & \boxed{0} \\ \boxed{0} & \boxed{1} & \boxed{1} & \boxed{-1} \end{matrix} \begin{matrix} 6 \\ 4 \end{matrix} \xrightarrow{\text{ReLU}} \begin{matrix} \boxed{6} \\ 4 \end{matrix} \approx \begin{matrix} \boxed{6} \\ 4 \end{matrix} \begin{matrix} Y_1 \\ Y_2 \end{matrix}$$



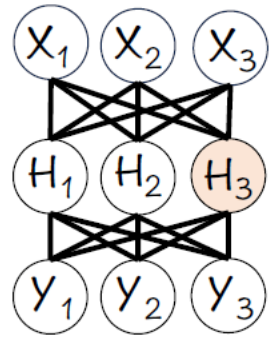
Exercise 2

$$\begin{matrix} \boxed{3} & \boxed{3} \end{matrix}$$

$$\begin{matrix} 1 \\ 0 \\ 2 \end{matrix} \begin{matrix} X_1 \\ X_2 \\ X_3 \end{matrix}$$

$$\begin{matrix} \boxed{0} & \boxed{1} & \boxed{1} & \boxed{-1} \\ \boxed{1} & \boxed{0} & \boxed{1} & \boxed{0} \\ \boxed{0} & \boxed{1} & \boxed{0} & \boxed{-1} \end{matrix} \xrightarrow{\text{ReLU}} \begin{matrix} \boxed{1} \\ 3 \end{matrix} \approx \begin{matrix} \boxed{1} \\ 3 \end{matrix} \begin{matrix} H_1 \\ H_2 \\ H_3 \end{matrix}$$

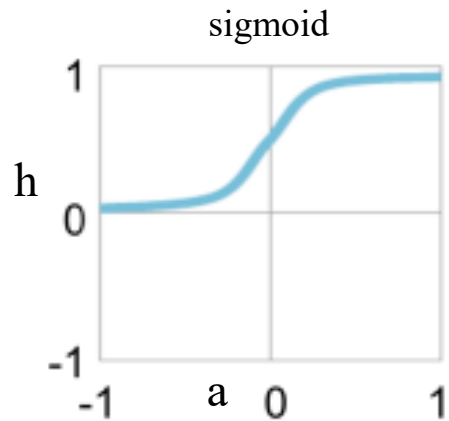
$$\begin{matrix} \boxed{1} & \boxed{1} & \boxed{0} & \boxed{0} \\ \boxed{0} & \boxed{-1} & \boxed{0} & \boxed{1} \\ \boxed{1} & \boxed{1} & \boxed{1} & \boxed{-1} \end{matrix} \begin{matrix} 4 \\ -2 \\ 3 \end{matrix} \xrightarrow{\text{ReLU}} \begin{matrix} \boxed{4} \\ 0 \\ 3 \end{matrix} \approx \begin{matrix} \boxed{4} \\ 0 \\ 3 \end{matrix} \begin{matrix} Y_1 \\ Y_2 \\ Y_3 \end{matrix}$$



Exercise 3

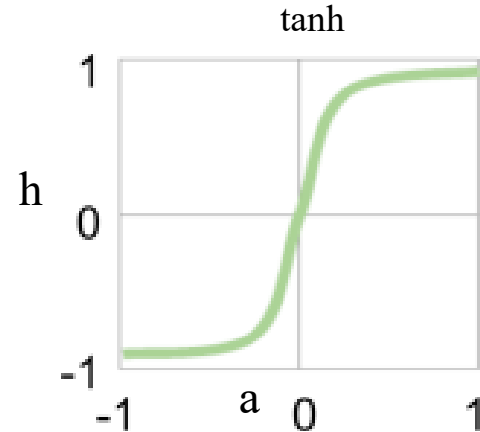
$$\begin{matrix} \boxed{-1} & \boxed{0} \end{matrix}$$

Activation Functions: Some Common Choices



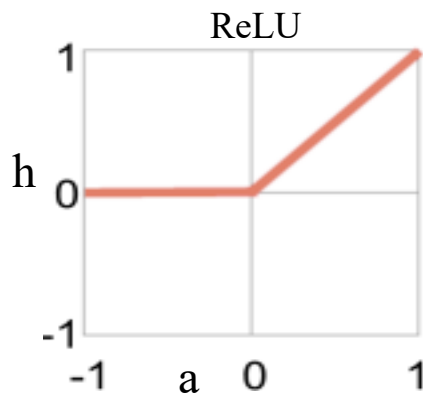
For sigmoid as well as tanh, gradients saturate (become close to zero as the function tends to its extreme values)

Sigmoid: $h = \sigma(a) = \frac{1}{1 + \exp(-a)}$



Preferred more than sigmoid. Helps keep the mean of the next layer's inputs close to zero (with sigmoid, it is close to 0.5)

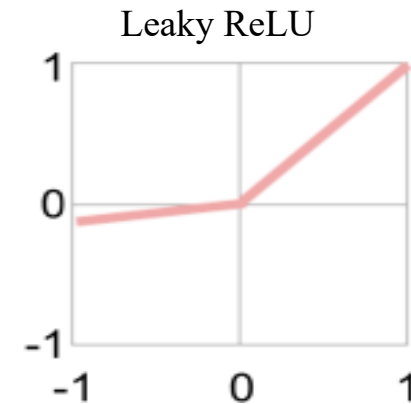
tanh (tan hyperbolic): $h = \frac{\exp(a) - \exp(-a)}{\exp(a) + \exp(-a)} = 2\sigma(2a) - 1$



ReLU and Leaky ReLU are among the most popular ones (also efficient to compute)

Helps fix the dead neuron problem of ReLU when a is a negative number

ReLU (Rectified Linear Unit): $h = \max(0, a)$



Without nonlinear activation, a deep neural network is equivalent to a linear model no matter how many layers we use

Leaky ReLU: $h = \max(\beta a, a)$
where β is a small positive number

Exercises: Activation Functions-ReLU, Sigmoid, Tanh

SORBONNE
UNIVERSITÉ

1

-1

0

1

-2

0

1

-3

0

3

2

x_1

x_2

1

-1

-3

ReLU

1

0

y_1

y_2

y_3

x_1

x_2

y_1

y_2

y_3

Exercise 1

0

1

-1

0

1

-2

0

1

-3

0

3

2

x_1

x_2

1

-1

-3

Sigmoid

0.5

0

y_1

y_2

y_3

x_1

x_2

y_1

y_2

y_3

Exercise 2

0.5

1

-1

-2

1

-2

-2

1

-3

-2

3

2

x_1

x_2

1

-1

-5

Tanh{3}

0

-1

y_1

y_2

y_3

x_1

x_2

y_1

y_2

y_3

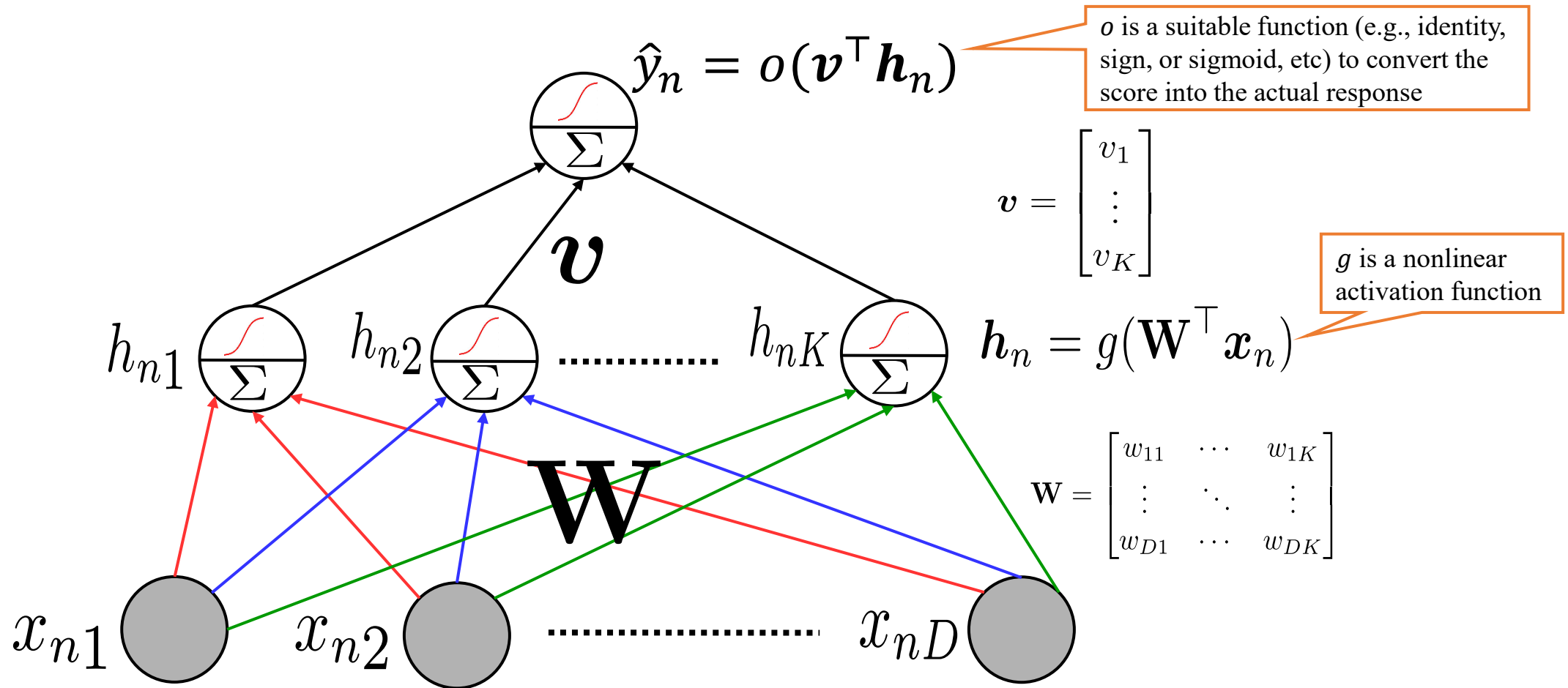
Exercise 3

-1

Examples of some basic NN/MLP architectures

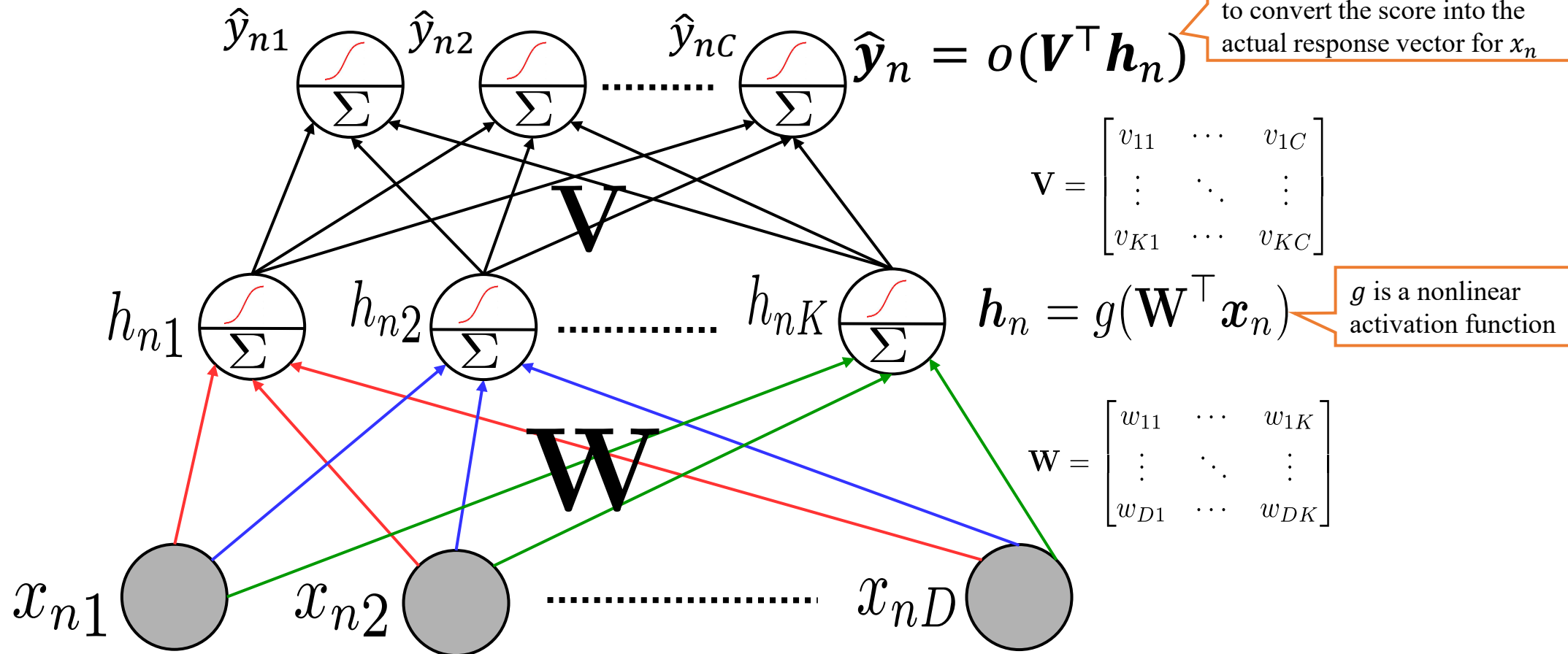
Single Hidden Layer and Single Output

- One hidden layer with K nodes and a single output (e.g., scalar-valued regression or binary classification)



Single Hidden Layer and Multiple Outputs

- One hidden layer with K nodes and a vector of C outputs (e.g., vector-valued regression or multi-class classification or multi-label classification)



Multiple Hidden Layers (One/Multiple Outputs)

- Most general case: Multiple hidden layers with (with same or different number of hidden nodes in each) and a scalar or vector-valued output

Each hidden layer uses a nonlinear activation function g (essential, otherwise the network can't learn nonlinear functions and reduces to a linear model)

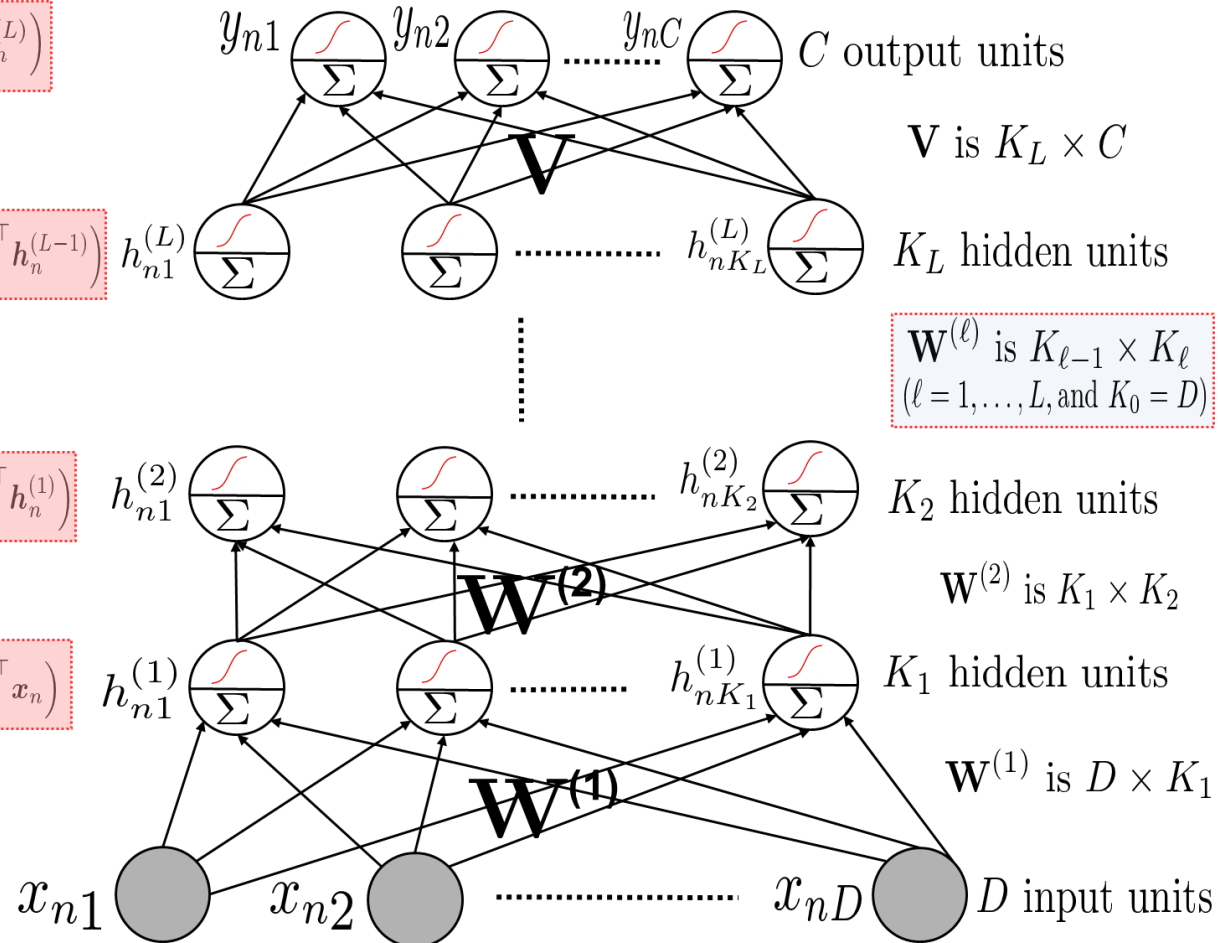
Note: Nonlinearity g is applied **element-wise** on its inputs so $h_n^{(\ell)}$ has the same size as vector $W^{(\ell)} h_n^{(\ell-1)}$

$$y_n = o(V^T h_n^{(L)})$$

$$h_n^{(L)} = g(W^{(L)T} h_n^{(L-1)})$$

$$h_n^{(2)} = g(W^{(2)T} h_n^{(1)})$$

$$h_n^{(1)} = g(W^{(1)T} x_n)$$



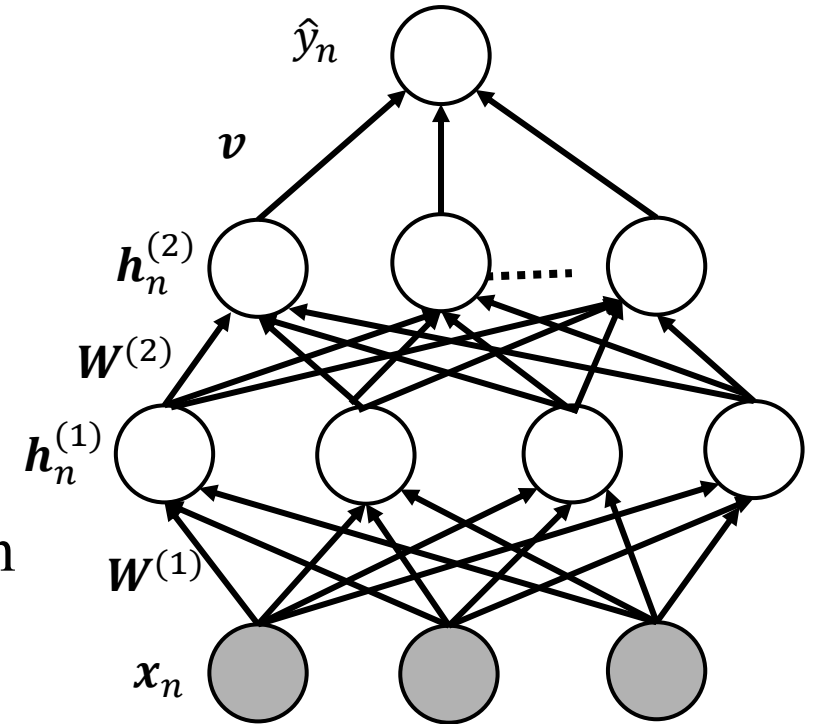
The Bias Term

- Each layer's pre-activations $\mathbf{z}_n^{(\ell)}$ have an add bias term $\mathbf{b}^{(\ell)}$ (has the same size as $\mathbf{z}_n^{(\ell)}$ and $\mathbf{h}_n^{(\ell)}$) as well

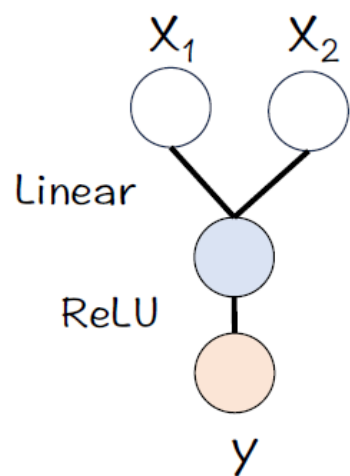
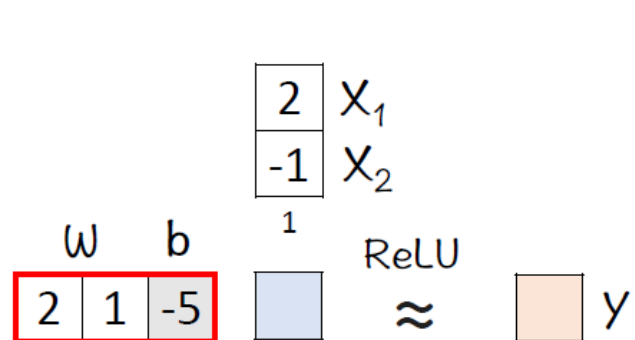
$$\mathbf{z}_n^{(\ell)} = \mathbf{W}^{(\ell)\top} \mathbf{x}_n^{(\ell-1)} + \mathbf{b}^{(\ell)}$$

$$\mathbf{h}_n^{(\ell)} = g(\mathbf{z}_n^{(\ell)})$$

- Bias term increases the expressiveness of the network and ensures that we have nonzero activations/pre-activations even if this layer's input is a vector of all zeros
- Note that the bias term is the same for all inputs (does not depend on n)
- The bias term $\mathbf{b}^{(\ell)}$ is also learnable

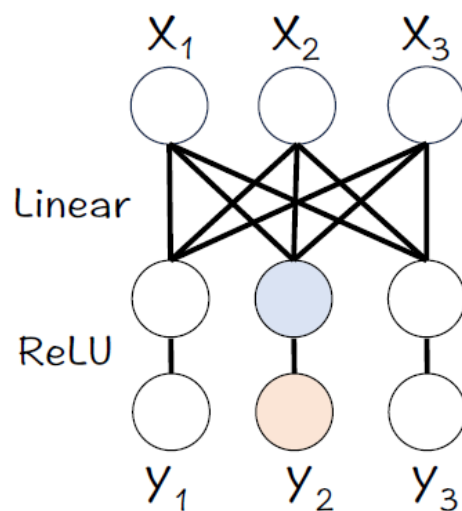
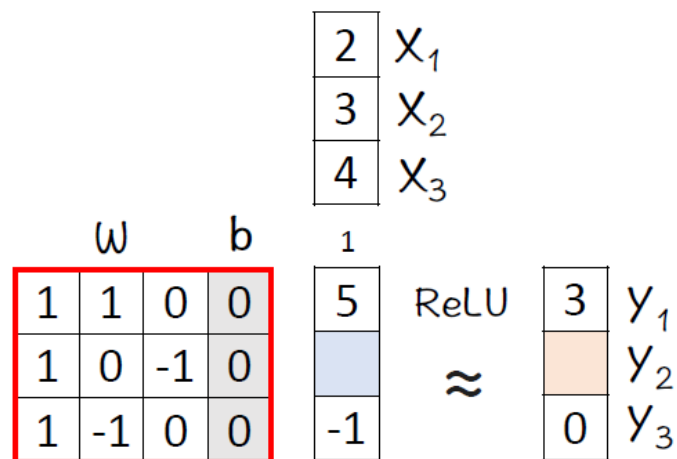


Exercises: Artificial Neuron



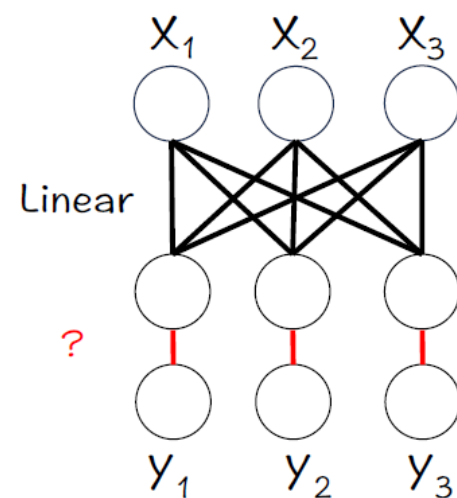
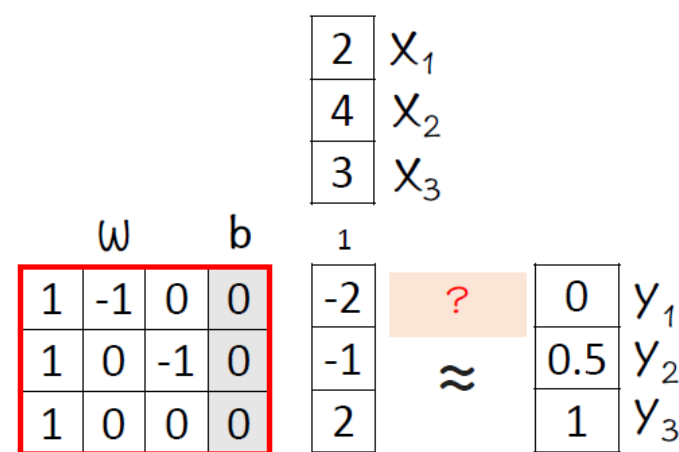
Exercise 1

-2
0



Exercise 2

-2
0

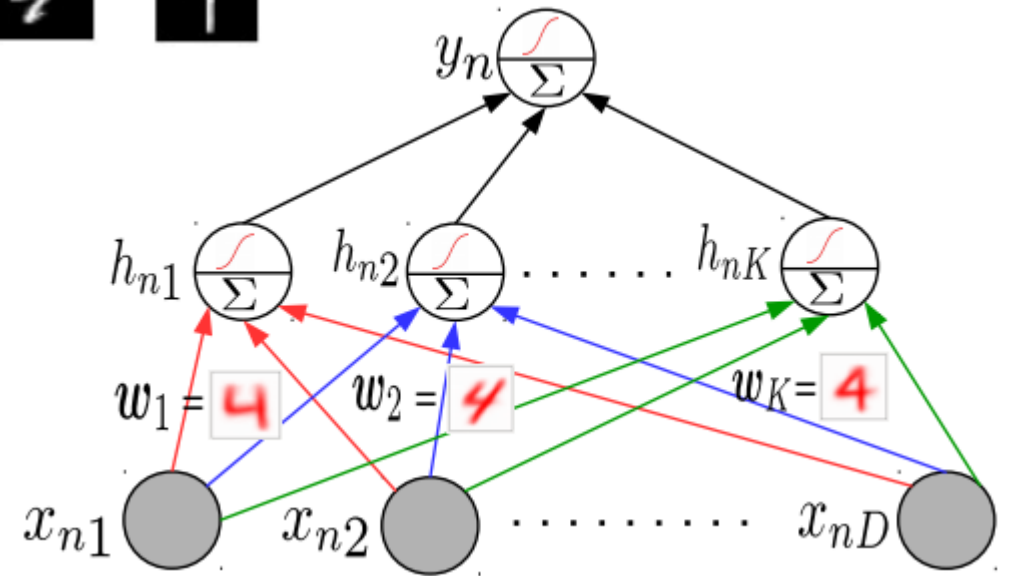
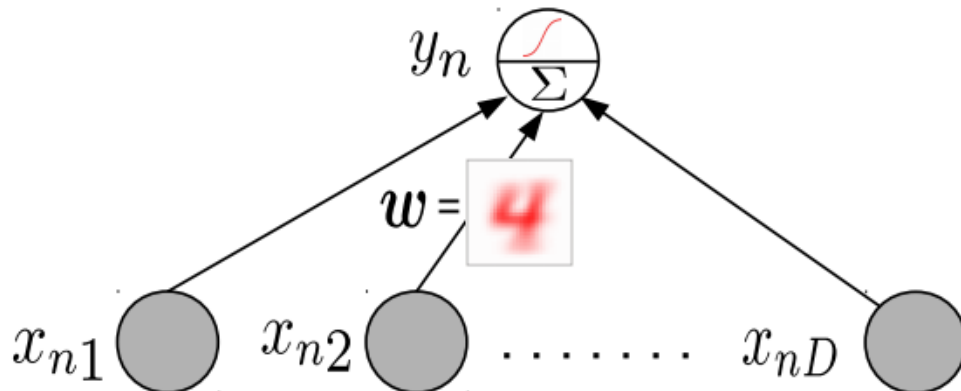


Exercise 3

Sigmoid

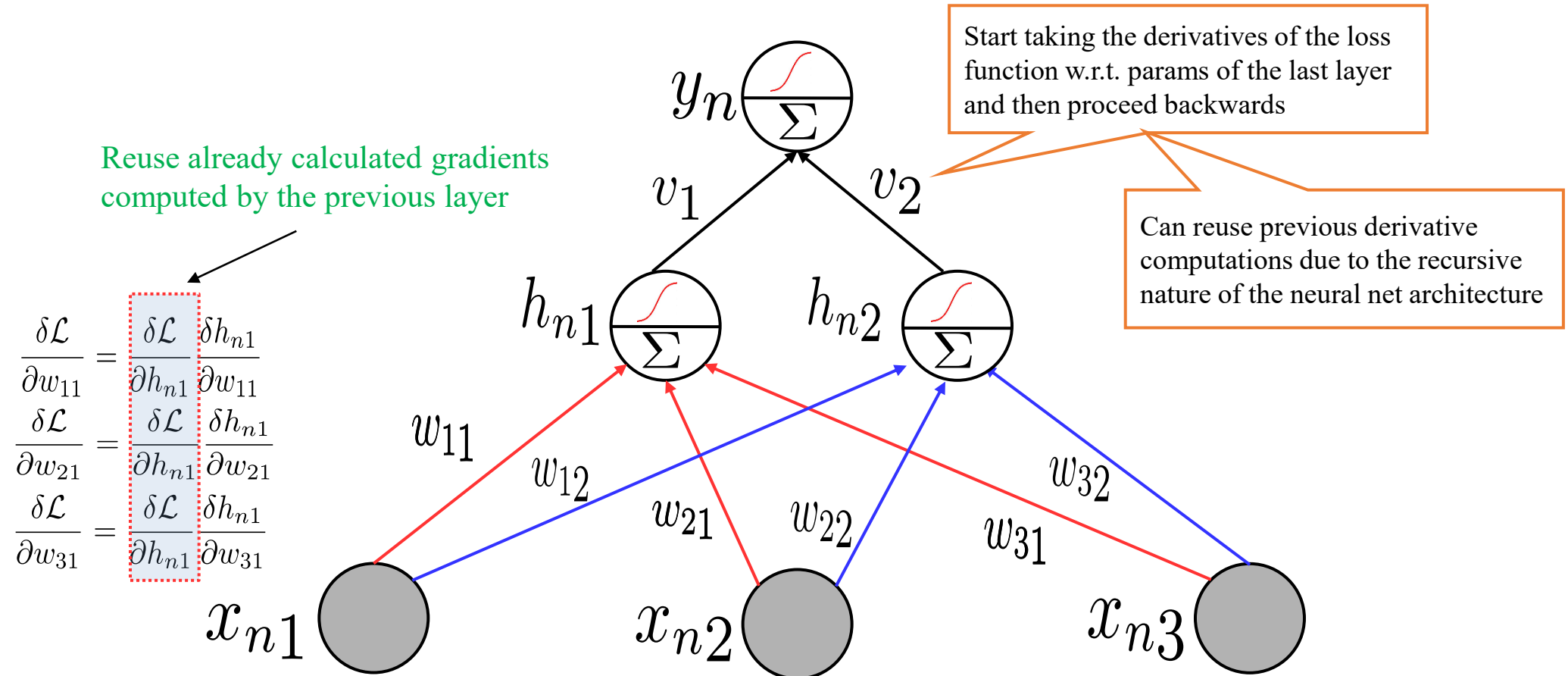
Why Neural Networks Work Better?

- Linear models tend to only learn the “average” pattern
 - E.g., Weight vector of a linear classification model represent average pattern of a class
- Deep models can learn multiple patterns (each hidden node can learn one pattern)
 - Thus, deep models can learn to capture more subtle variations that a simpler linear model



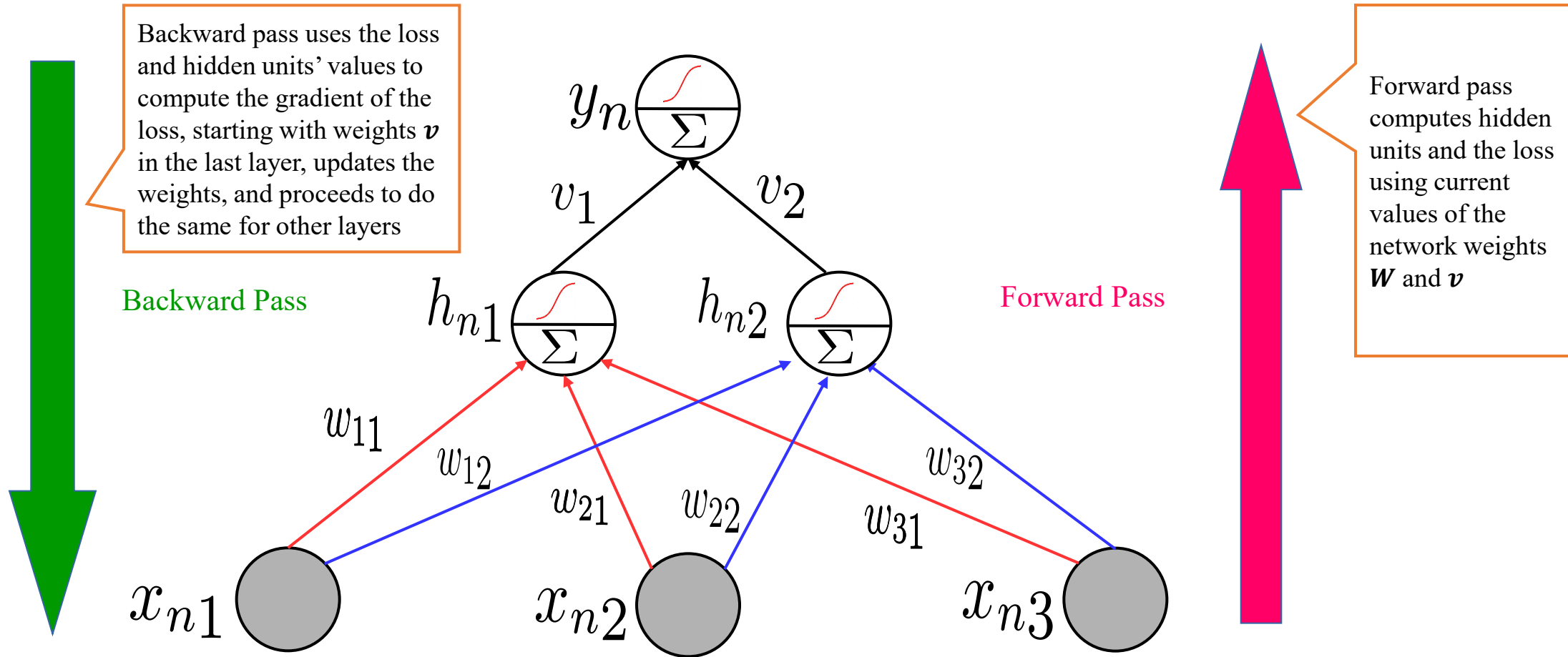
Backpropagation

- Backpropagation = Gradient descent using chain rule of derivatives
- Chain rule of derivatives: Example, if $y = f_1(x)$ and $x = f_2(z)$ then $\frac{\partial y}{\partial z} = \frac{\partial y}{\partial x} \frac{\partial x}{\partial z}$



Backpropagation

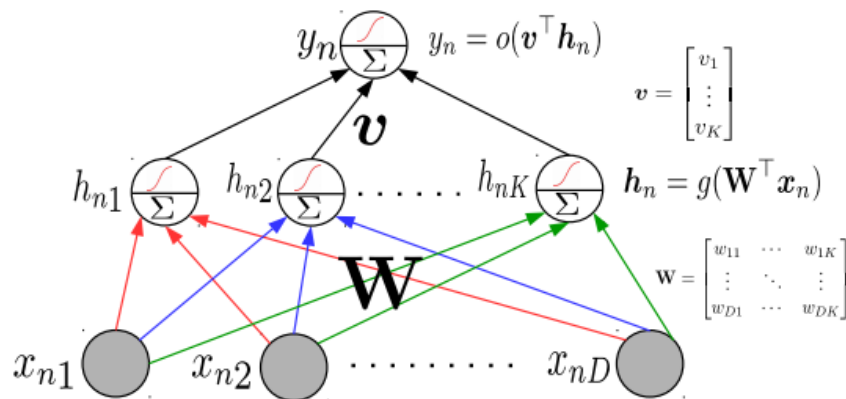
- Backprop iterates between a forward pass and a backward pass



- Software frameworks such as Tensorflow and PyTorch support this

Backpropagation through an example

Consider a single hidden layer MLP



Assuming regression ($o = \text{identity}$),
the loss function for this model

$$\begin{aligned}\mathcal{L} &= \frac{1}{2} \sum_{n=1}^N \left(y_n - \mathbf{v}^\top \mathbf{h}_n \right)^2 \\ &= \frac{1}{2} \sum_{n=1}^N \left(y_n - \sum_{k=1}^K v_k h_{nk} \right)^2 \\ &= \frac{1}{2} \sum_{n=1}^N \left(y_n - \sum_{k=1}^K v_k g(\mathbf{w}_k^\top \mathbf{x}_n) \right)^2\end{aligned}$$

- To use gradient methods for $\mathbf{W}$, $\mathbf{v}$, we need gradients.

- Gradient of $\mathcal{L}$ w.r.t. $\mathbf{v}$ is straightforward

$$\frac{\partial \mathcal{L}}{\partial v_k} = - \sum_{n=1}^N \left(y_n - \sum_{k=1}^K v_k g(\mathbf{w}_k^\top \mathbf{x}_n) \right) h_{nk} = \sum_{n=1}^N \mathbf{e}_n h_{nk}$$

- Gradient of $\mathcal{L}$ w.r.t. $\mathbf{W}$ requires chain rule

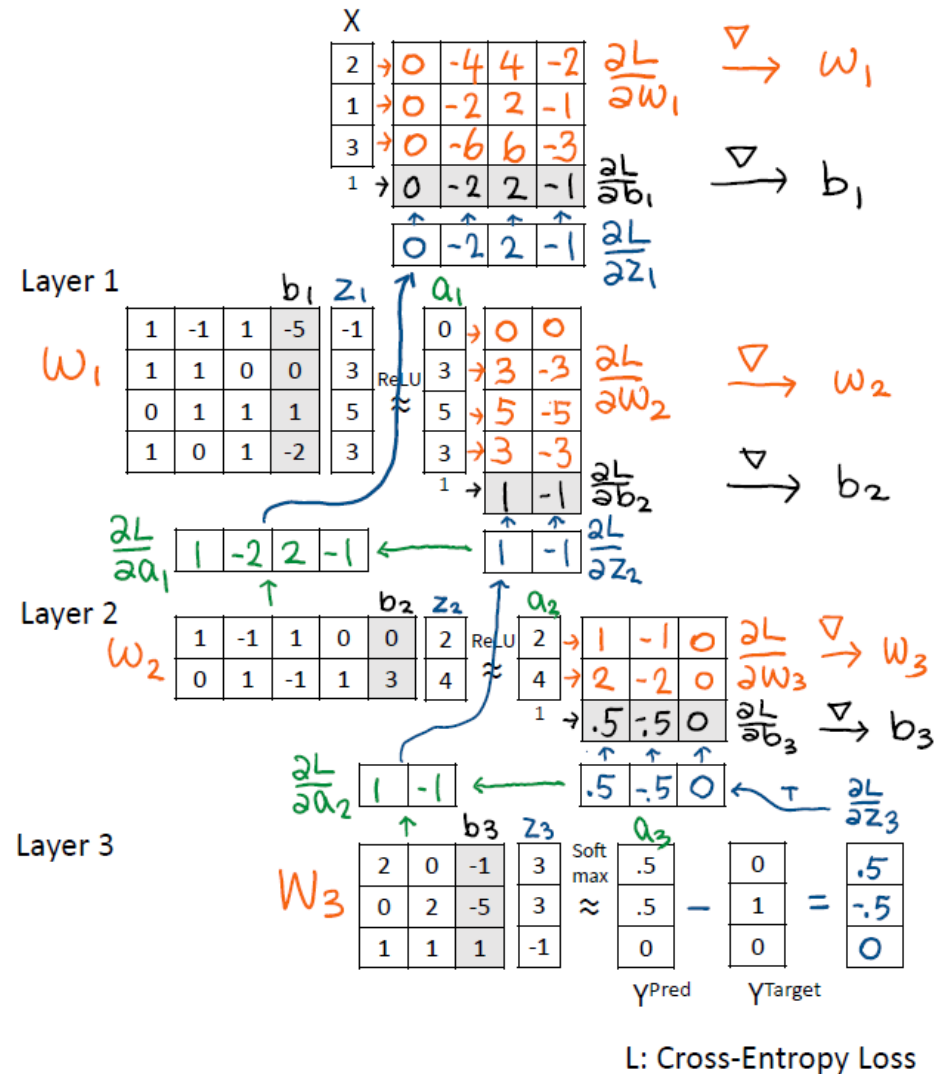
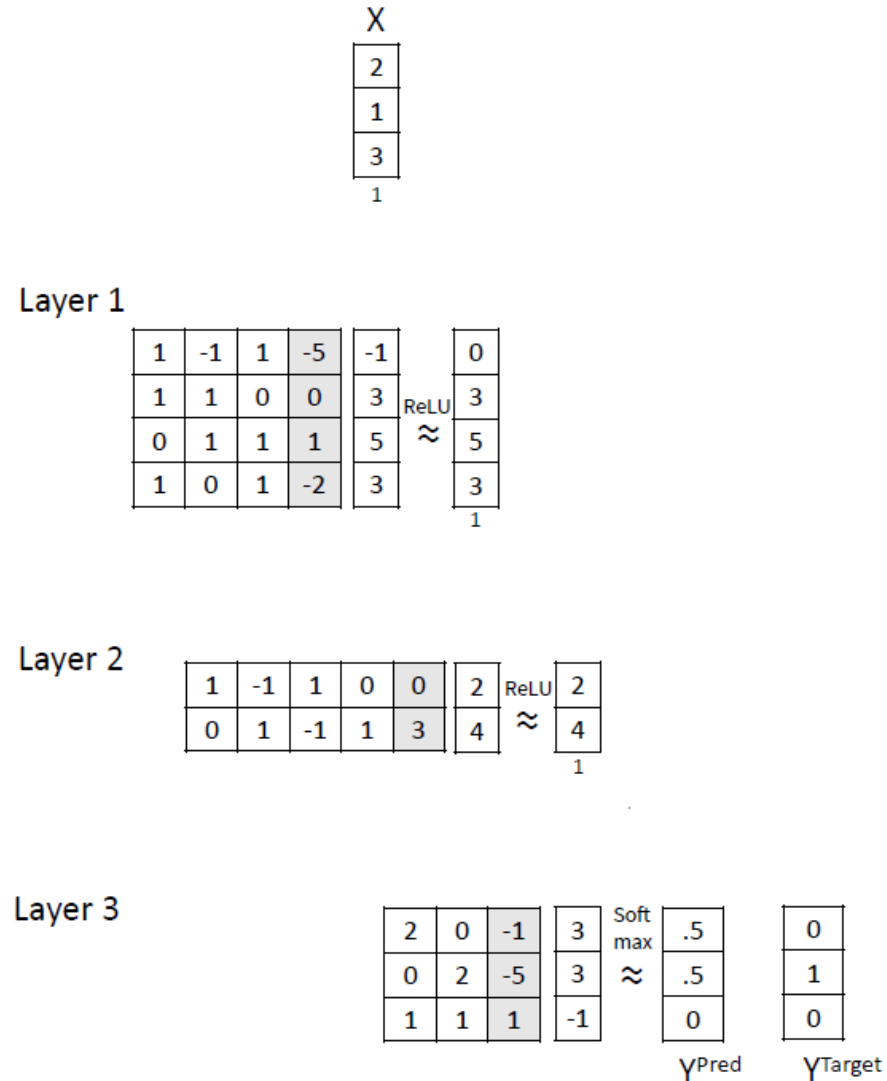
$$\frac{\partial \mathcal{L}}{\partial w_{dk}} = \sum_{n=1}^N \frac{\partial \mathcal{L}}{\partial h_{nk}} \frac{\partial h_{nk}}{\partial w_{dk}}$$

$$\frac{\partial \mathcal{L}}{\partial h_{nk}} = - \left(y_n - \sum_{k=1}^K v_k g(\mathbf{w}_k^\top \mathbf{x}_n) \right) v_k = - \mathbf{e}_n v_k$$

$$\frac{\partial h_{nk}}{\partial w_{dk}} = g'(\mathbf{w}_k^\top \mathbf{x}_n) x_{nd} \quad (\text{note: } h_{nk} = g(\mathbf{w}_k^\top \mathbf{x}_n))$$

- **Forward prop** computes errors $\mathbf{e}_n$ using current $\mathbf{W}$, $\mathbf{v}$.
Backprop updates NN params $\mathbf{W}$, $\mathbf{v}$ using grad methods
- Backprop caches many of the calculations for reuse

Exercise: Backpropagation



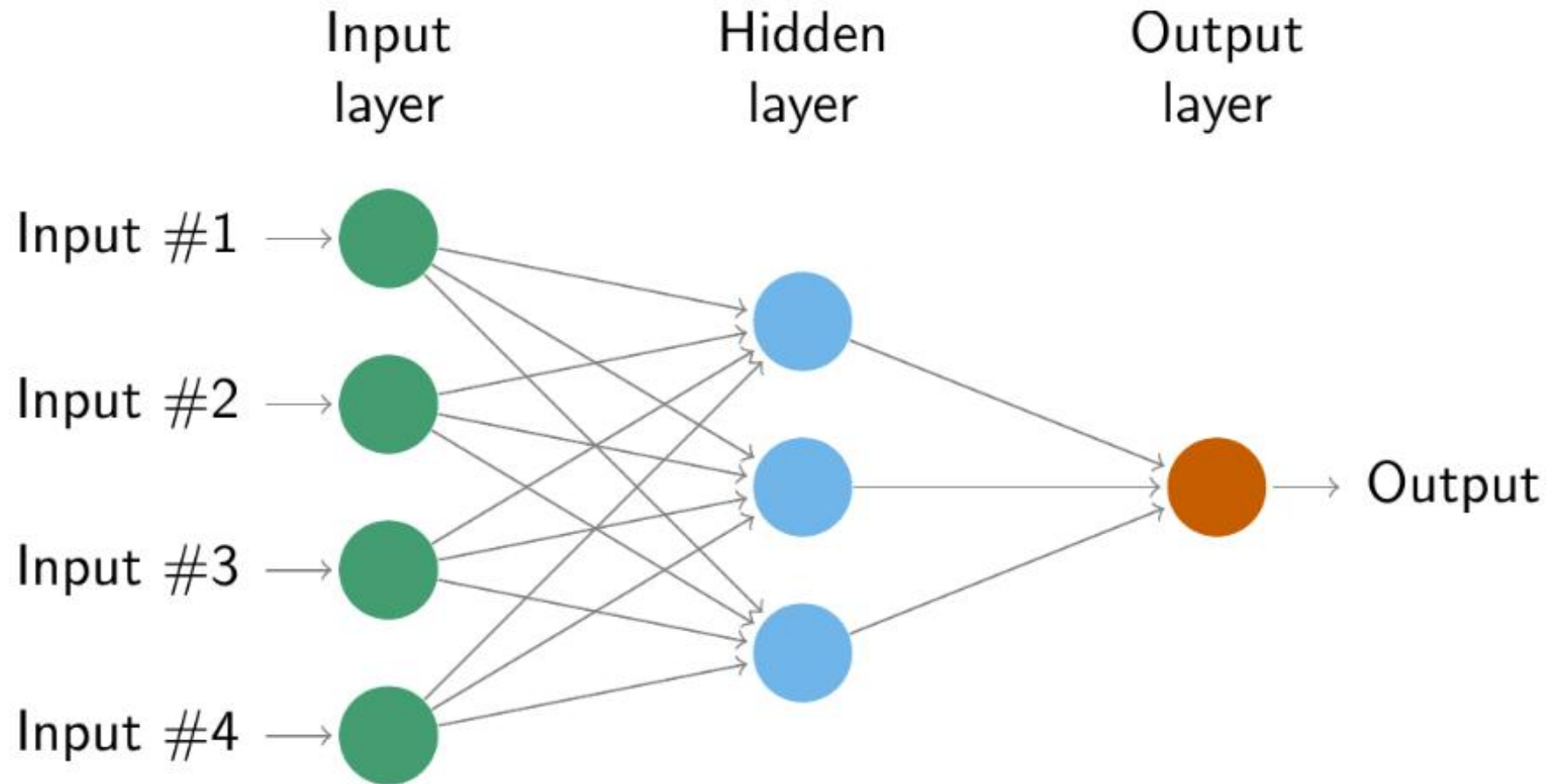
Exercise: Backpropagation

Steps:

1. **Forward Pass:** Given a multi layer perceptron (3 levels), an input vector X , predictions $Y^{Pred} = [0.5, 0.5, 0]$, and ground truth label $Y^{Target} = [0, 1, 0]$.
2. **Backpropagation:** Insert cells to hold our calculations.
3. **Layer 3** - Softmax (blue): Calculate $\partial L / \partial z_3$ directly using the simple equation: $Y^{Pred} - Y^{Target} = [0.5, -0.5, 0]$. This simple equation is the benefit of using Softmax and Cross Entropy Loss together.
4. **Layer 3** - Weights (orange) & Biases (black): Calculate $\partial L / \partial W_3$ and $\partial L / \partial b_3$ by multiplying $\partial L / \partial z_3$ and $[a_2 | 1]$.
5. **Layer 2** - Activations (green): Calculate $\partial L / \partial a_2$ by multiplying $\partial L / \partial z_3$ and W_3 .
6. **Layer 2** - ReLU (blue): Calculate $\partial L / \partial z_2$ by multiplying $\partial L / \partial a_2$ with 1 for positive values and 0 otherwise.
7. **Layer 2** - Weights (orange) & Biases (black): Calculate $\partial L / \partial W_2$ and $\partial L / \partial b_2$ by multiplying $\partial L / \partial z_2$ and $[a_1 | 1]$.
8. **Layer 1** - Activations (green): Calculate $\partial L / \partial a_1$ by multiplying $\partial L / \partial z_2$ and W_2 .
9. **Layer 1** - ReLU (blue): Calculate $\partial L / \partial z_1$ by multiplying $\partial L / \partial a_1$ with 1 for positive values and 0 otherwise.
10. **Layer 1** - Weights (orange) & Biases (black): Calculate $\partial L / \partial W_1$ and $\partial L / \partial b_1$ by multiplying $\partial L / \partial z_1$ and $[x | 1]$.
11. **Gradient Descent:** Update weights and biases (typically a learning rate is applied here).

Autoregressive Neural Network

Feed-forward Neural Network

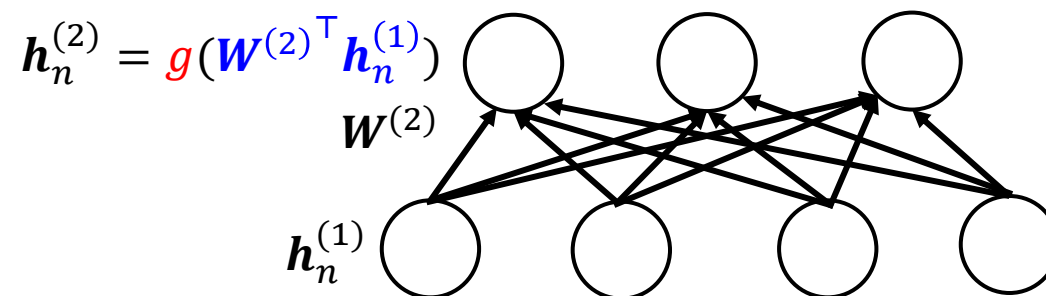
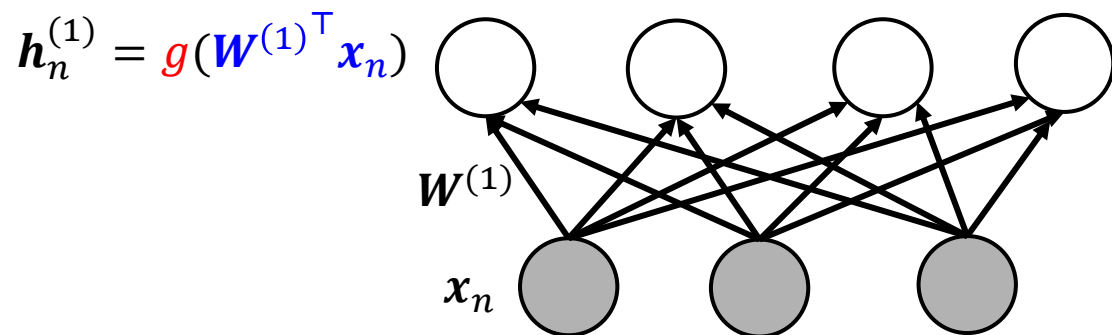


ARNN for Forecasting

- With time series data, lagged values of the time series can be used as inputs to a neural network, typically named as **Autoregressive neural network (ARNN)**
- $\text{ARNN}(p, k)$ is a feed-forward network with one hidden layer, with p lagged inputs and k nodes in the hidden layer.
- $\text{ARNN}(p, 0)$ model is equivalent to an $\text{ARIMA}(p, 0, 0)$ model, but without the restrictions on the parameters to ensure stationarity.
- With seasonal data, it is useful to also add the last observed values from the same season as inputs.
- For time series, the default is the optimal number of lags (according to the AIC) for a linear $\text{AR}(p)$ model. If k is not specified, it is set to $k = (p + 1)/2$ (rounded to the nearest integer).
- When it comes to forecasting, the network is applied iteratively.

Limitations/Shortcomings of MLP & ARNN

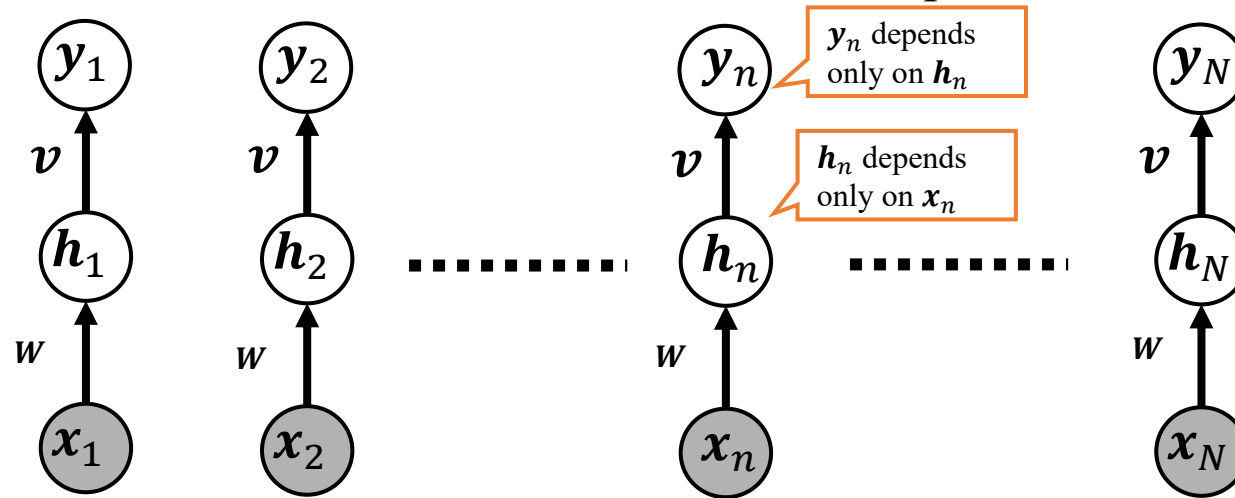
- MLP uses fully connected layers defined by matrix multiplications + nonlinearity



- MLP **ignores structure** (e.g., spatial/sequential) in the inputs
 - Not ideal for data such as images, text, etc. which are flattened as vectors when used with MLP
- Fully connected nature of MLP requires massive number of weights
 - Recall that each layer is fully connected so each layer needs a massive number of weights!

Recurrent Connections in Deep Neural Networks

- Feedforward nets such as MLP assume independent observations

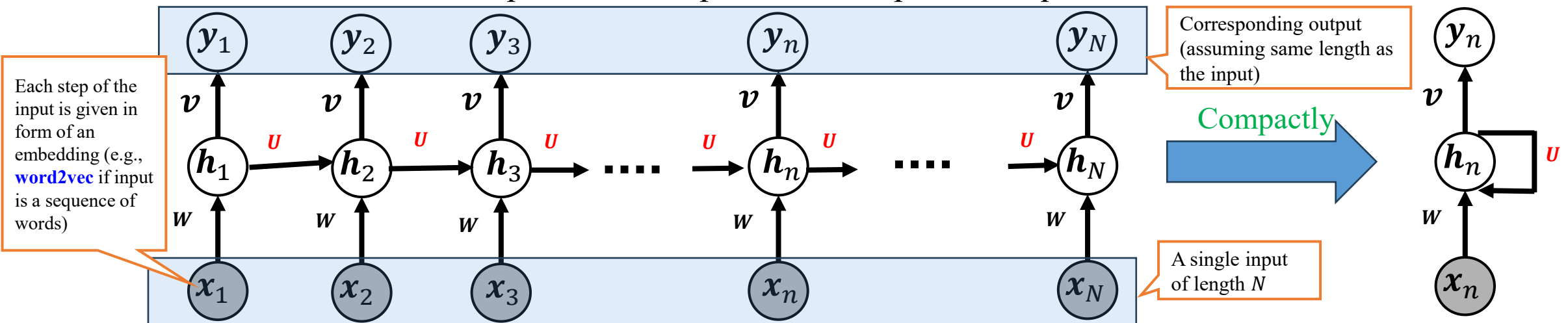


y_n depends only on h_n

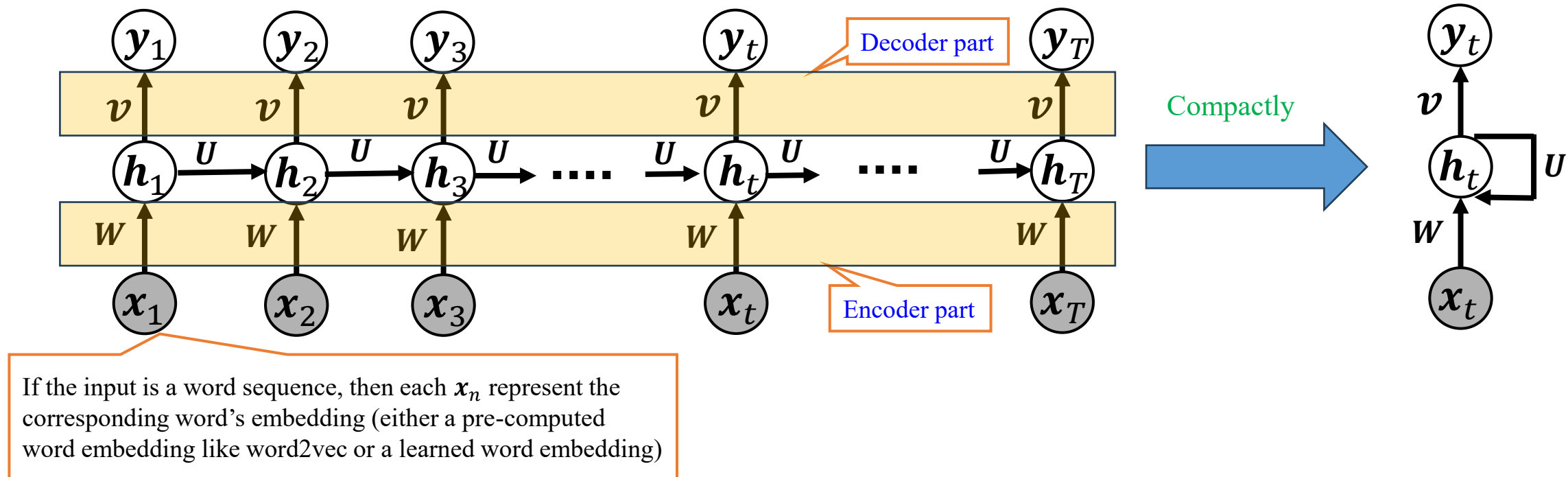
h_n depends only on x_n

Feedforward neural networks are not ideal when inputs $[x_1, x_2, \dots, x_N]$ and/or outputs $[y_1, y_2, \dots, y_N]$ represent sequential data (e.g., sequence of words, video (sequence of frames), etc).

- A **recurrent structure** can be helpful if each input and/or output is a sequence



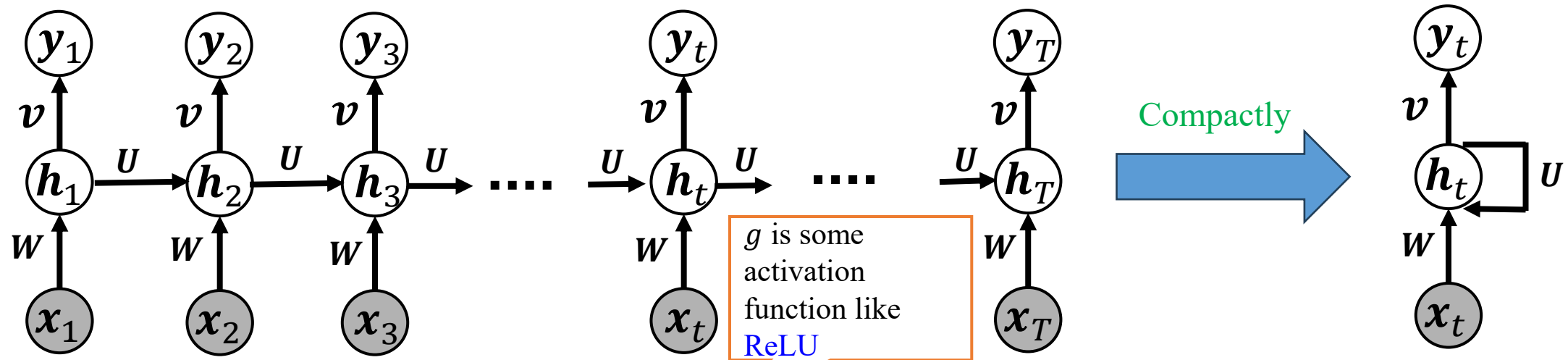
- RNNs are used when each input or output or both are **sequences of tokens**



- Hidden state h_t is supposed to remember everything up to time $t - 1$. However, in practice, RNNs have difficulties remembering the distant past
 - Variants such as LSTM, GRU, etc mitigate this issue to some extent
- Slow processing is another major issue (e.g., can't compute h_t before computing h_{t-1})

Recurrent Neural Networks

- A basic RNN's architecture (assuming input and output sequence have same lengths)
- RNN has three sets of weights W, U, v



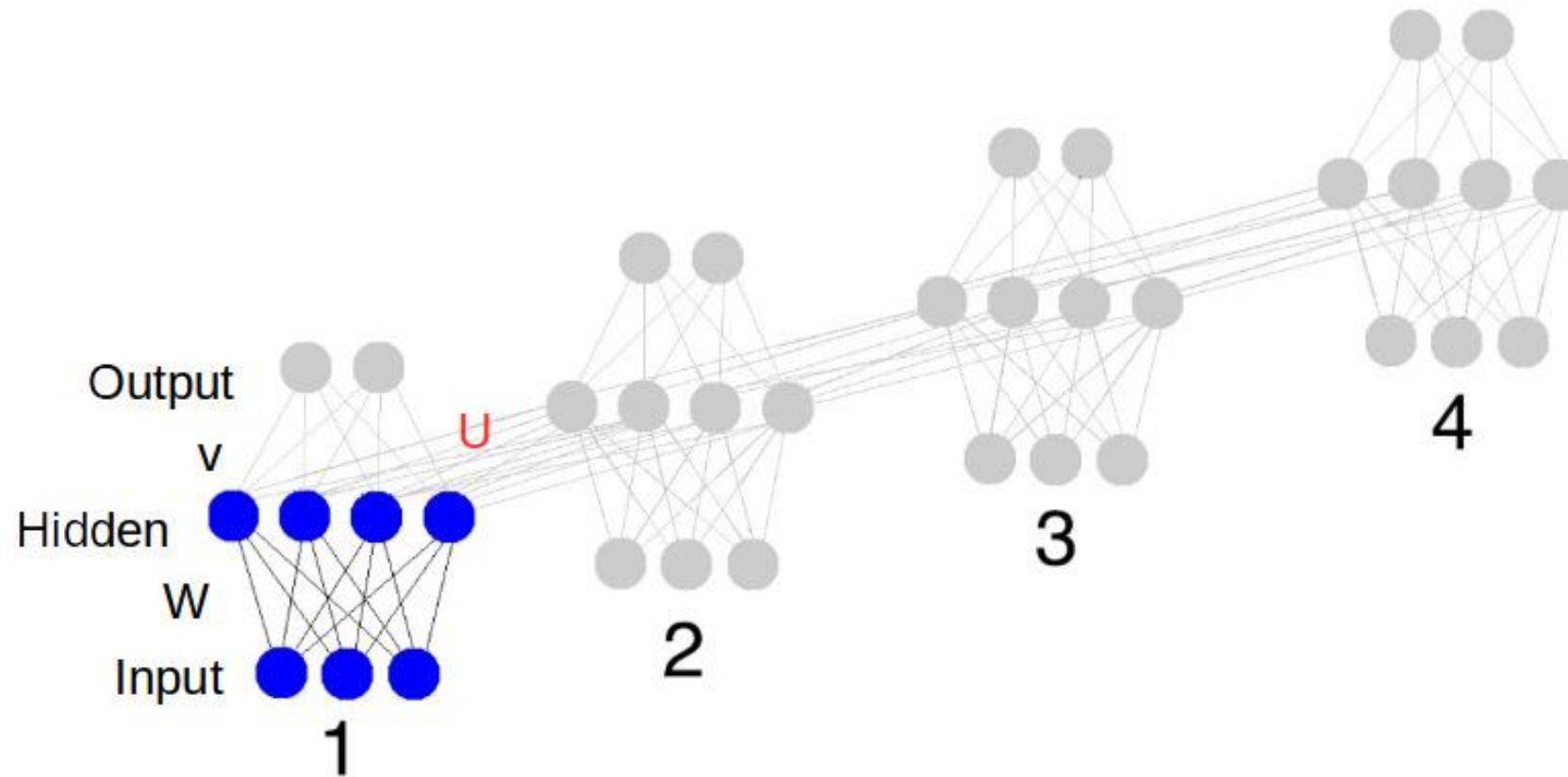
- W and U model how h_t at step t is computed: $h_t = g(Wx_t + Uh_{t-1})$
- v models the hidden layer to output mapping, e.g., $y_t = o(vh_t)$
- **Important:** Same W, U, v are used at all steps of the sequence (weight sharing)

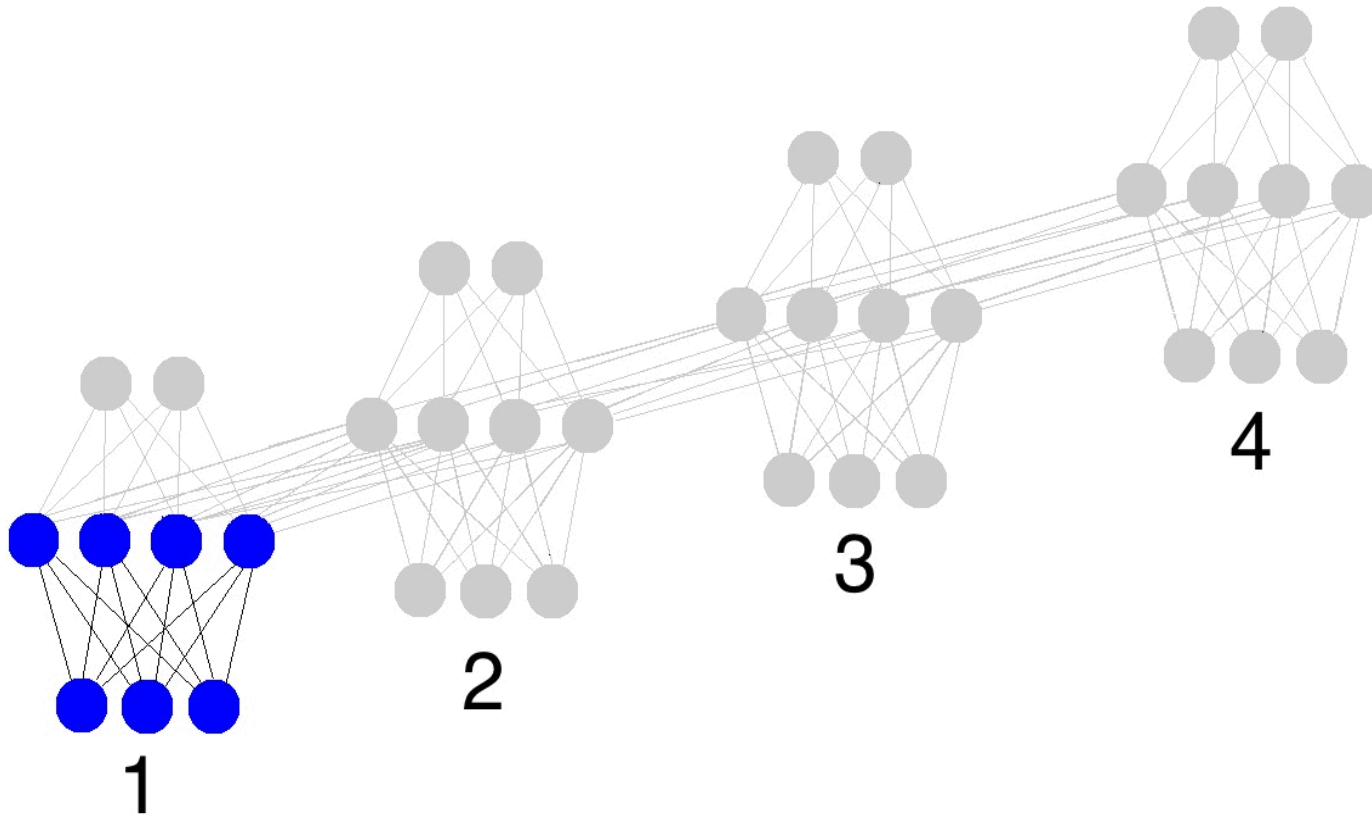
o depends on the nature of y_t . If it is categorical then o can be **softmax**

Given in form of an embedding
(e.g., word embedding if x_1 is a word)

Recurrent Neural Nets (RNN)

- A more “micro” view of RNN (the transition matrix U connects the hidden states across observations, propagating information along the sequence)





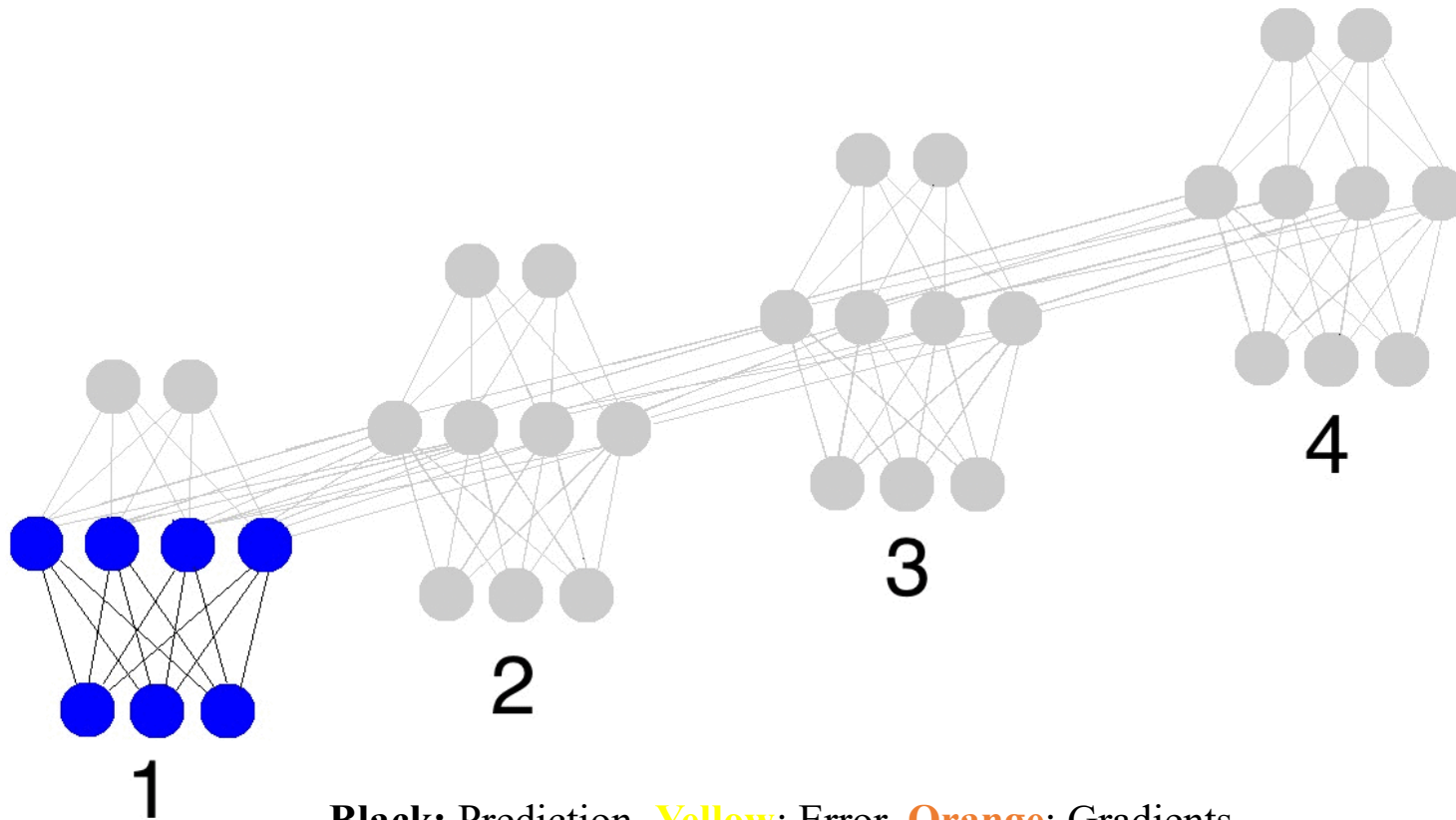
MakeAGIF.com

Workflow of RNN:

- The gif above reflects the magic of recurrent networks.
- It depicts 4 timesteps. The first is exclusively influenced by the input data.
- The second one is a mixture of the first and second inputs. This continues on.
- You should recognize that, in some way, network 4 is "full".
- Presumably, timestep 5 would have to choose which memories to keep and which ones to overwrite.
- This is very real. It's the notion of memory "capacity".
- As you might expect, bigger layers can hold more memories for a longer period of time.

Training RNN

- Trained using **Backpropagation Through Time** (forward propagate from step 1 to end, and then backward propagate from end to step)
- Think of the time-dimension as another hidden layer and then it is just like standard backpropagation for feedforward neural nets

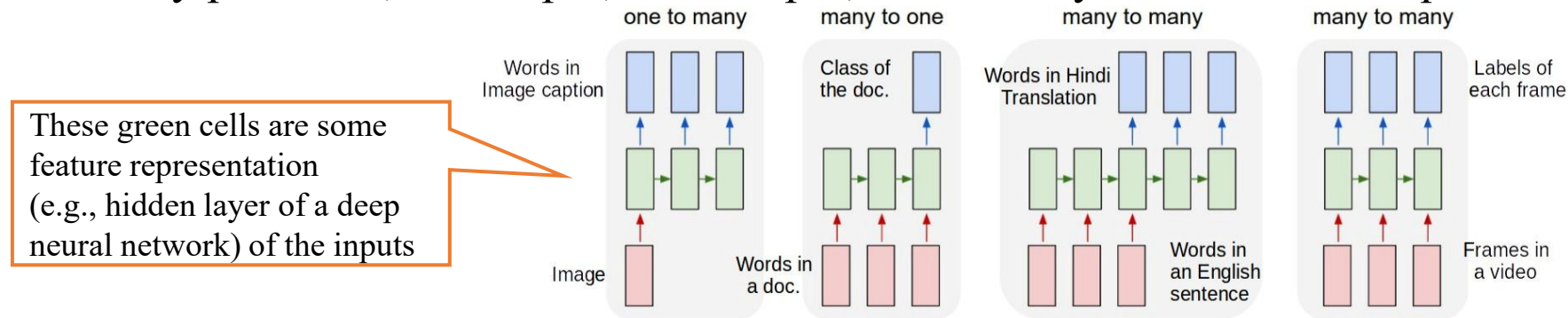


Black: Prediction, **Yellow:** Error, **Orange:** Gradients

- They learn by fully propagating forward from 1 to 4 (through an entire sequence of arbitrary length), and then backpropagating all the derivatives from 4 back to 1.
- You can also pretend that it's just a funny shaped normal neural network, except that we're re-using the same weights (synapses 0,1,and h) in their respective places.
- Other than that, it's normal backpropagation.

Pic source: <https://iamtrask.github.io/>

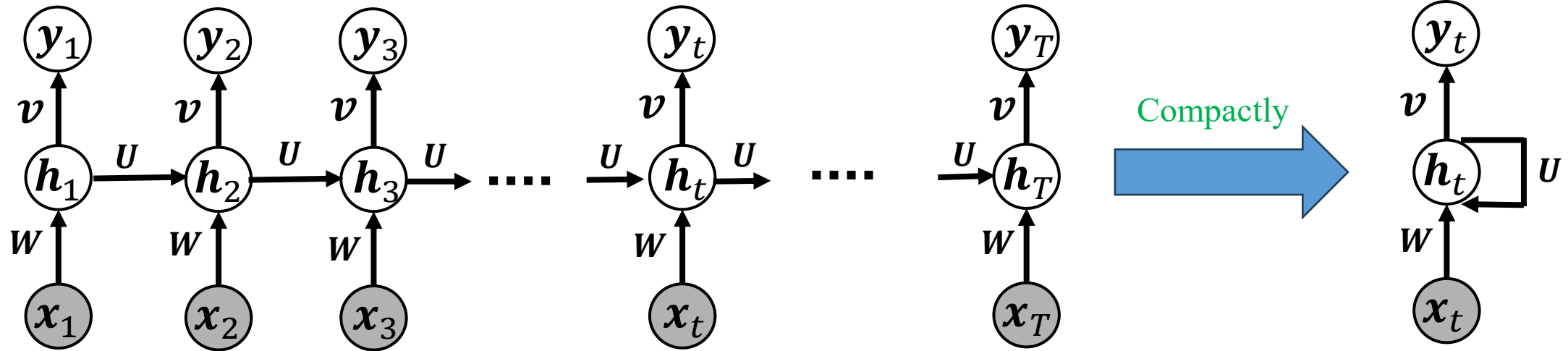
- In many problems, each input, each output, or both may be in form of sequences



- Different inputs or outputs need not have the same length
- Some examples of prediction tasks in such problems
 - **Image captioning:** Input is image (not a sequence), output is the caption (word sequence)
 - **Document classification:** Input is a word sequence, output is a categorical label
 - **Machine translation:** Input is a word sequence, output is a word sequence (in different language)
 - **Stock price prediction:** Input is a sequence of stock prices, output is its predicted price tomorrow
 - No input – just output (e.g., **generation** of random but plausible-looking text)

RNNs: Long Distant Past is Hard to Remember

- The hidden layer nodes h_t are supposed to summarize the past up to time $t - 1$



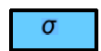
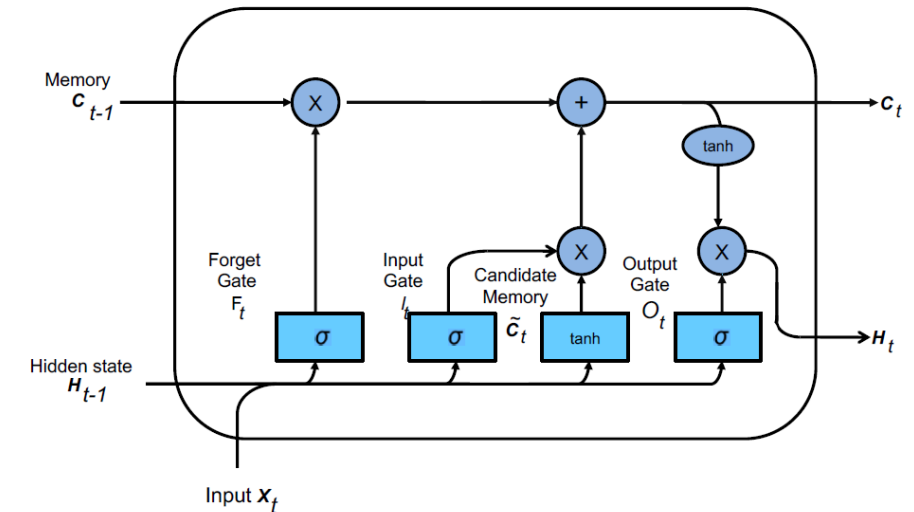
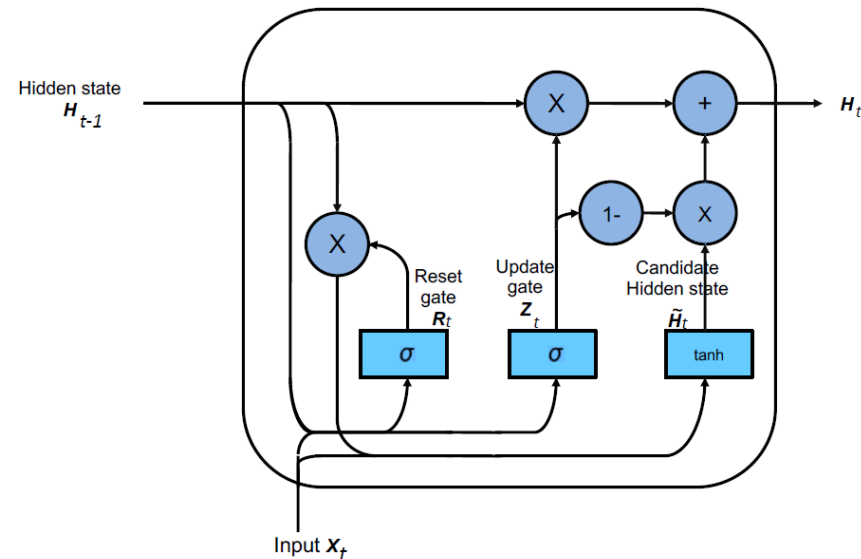
- In theory, they should. In practice, they can't. Some reasons
 - Vanishing gradients along the sequence too (due to repeated multiplications)
 - past knowledge gets “diluted”
 - Hidden nodes also have limited capacity because of their finite dimensionality
- Various extensions of RNNs have been proposed to address forgetting
 - Gated Recurrent Units (GRU), Long Short Term Memory (LSTM)

GRU and LSTM

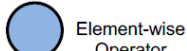
- Essentially an RNN, except that the hidden states are computed differently
- Recall that RNN computes the hidden states as

$$\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{x}_t + \mathbf{U}\mathbf{h}_{t-1})$$

- For RNN: State update is multiplicative (weak memory and gradient issues)
- GRU and LSTM contain specialized units and “memory” which modulate what/how much information from the past to retain/forget



FC layer with
Activation function



Element-wise
Operator

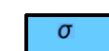


Copy

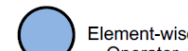


Concatenate

Pic source: <https://d2l.ai/>



FC layer with
Activation function



Element-wise
Operator



Copy



Concatenate

- In contrast, LSTM maintains a “context” $\mathbf{C}_t$ and computes hidden states as

$$\hat{\mathbf{C}}_t = \tanh(\mathbf{W}^c \mathbf{x}_t + \mathbf{U}^c \mathbf{h}_{t-1}) \quad (\text{“local” context, only up to immediately preceding state})$$

$$i_t = \sigma(\mathbf{W}^i \mathbf{x}_t + \mathbf{U}^i \mathbf{h}_{t-1}) \quad (\text{how much to take in the local context})$$

$$f_t = \sigma(\mathbf{W}^f \mathbf{x}_t + \mathbf{U}^f \mathbf{h}_{t-1}) \quad (\text{how much to forget the previous context})$$

$$o_t = \sigma(\mathbf{W}^o \mathbf{x}_t + \mathbf{U}^o \mathbf{h}_{t-1}) \quad (\text{how much to output})$$

$$\mathbf{C}_t = \mathbf{C}_{t-1} \odot f_t + \hat{\mathbf{C}}_t \odot i_t \quad (\text{a modulated additive update for context})$$

$$\mathbf{h}_t = \tanh(\mathbf{C}_t) \odot o_t \quad (\text{transform context into state and selectively output})$$

- Note: $\odot$ represents elementwise vector product. Also, state updates now additive, not multiplicative. Training using backpropagation through time.
- Many variants of LSTM exists, e.g., using $\mathbf{C}_t$ in local computations, Gated Recurrent Units (GRU), etc. Mostly minor variations of basic LSTM above.

Exercise: RNN

Recurrent Neural Network (RNN)

Input Sequence X

3	4	5	6
---	---	---	---

Parameters A

1	-1
1	1

 B

1
2

 C

-1	1
----	---

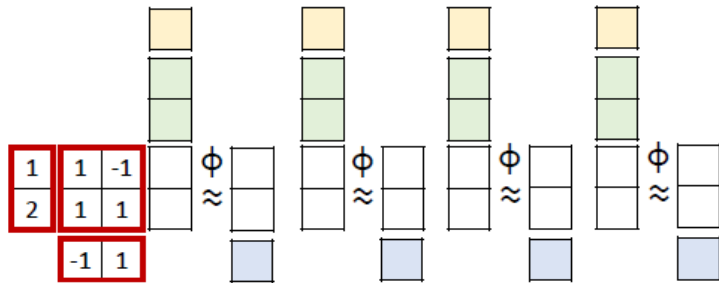
Activation Function ϕ : ReLU

Hidden States H_0

0
0

Output Sequence Y

--	--	--	--



Recurrent Neural Network (RNN)

Input Sequence X

3	4	5	6
---	---	---	---

Parameters A

1	-1
1	1

 B

1
2

 C

-1	1
----	---

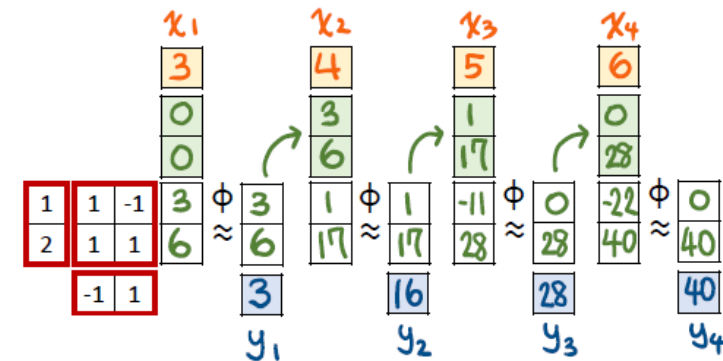
Activation Function ϕ : ReLU

Hidden States H_0

0
0

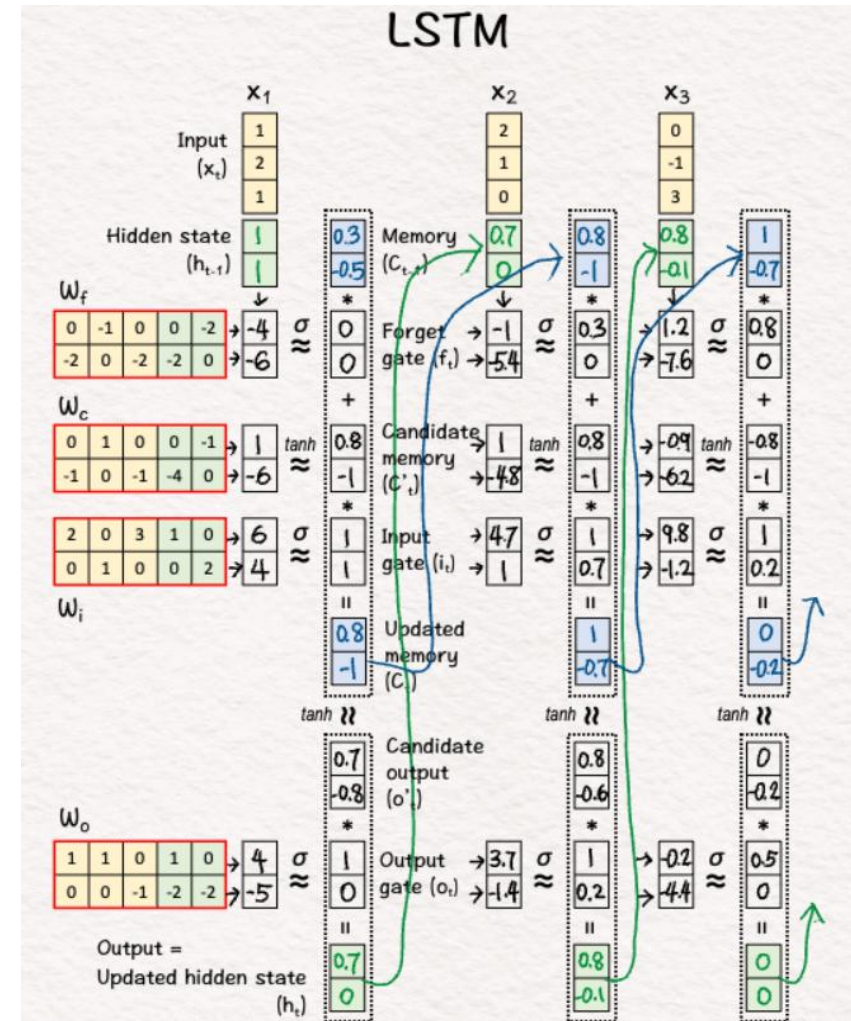
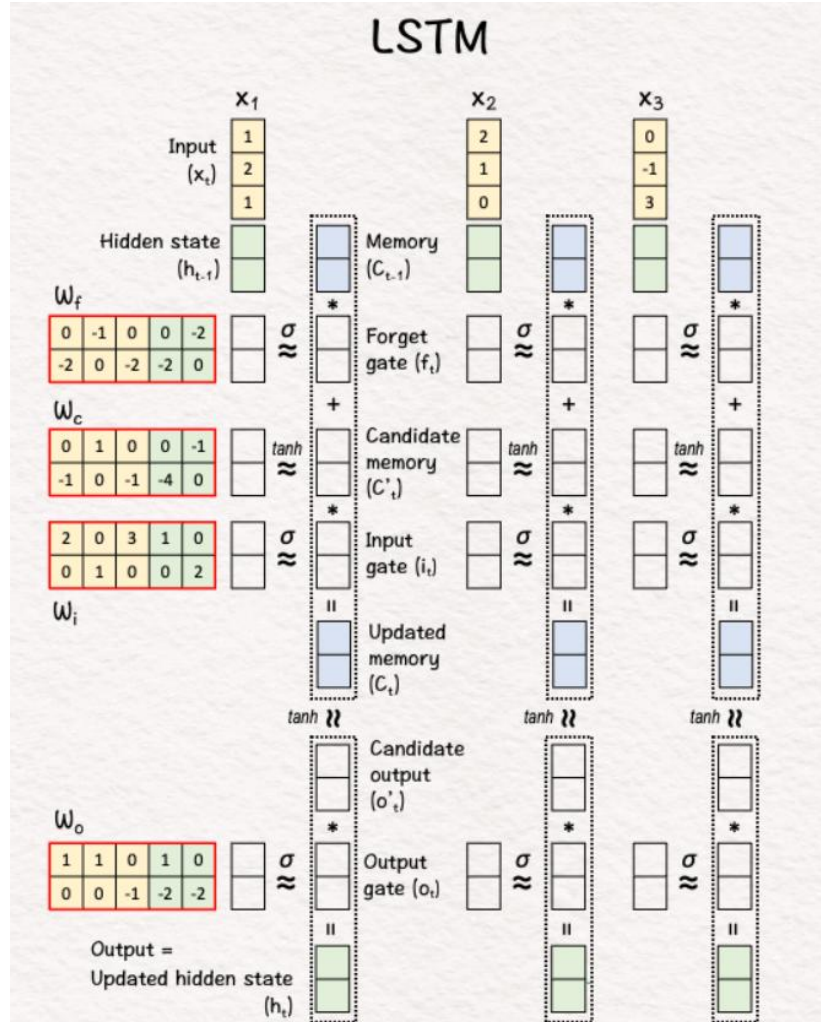
Output Sequence Y

--	--	--	--



Exercise: LSTM

Initialize: Randomly set the previous hidden state h_0 to $[1, 1]$ and memory cells C_0 to $[0.3, -0.5]$



SOTA Forecasting Methods

Pretrained Models

Single model used across all tasks

- LLMTime
- ForecastPFN
- Lag-Llama
- Monai
- Chronos

Task-specific Models

Separate model fine-tuned for each task

- PatchTST
- DeepAR
- EWNet
- TFT
- DLinear
- NBEATS
- NHITS
- CatBoost

Local Models

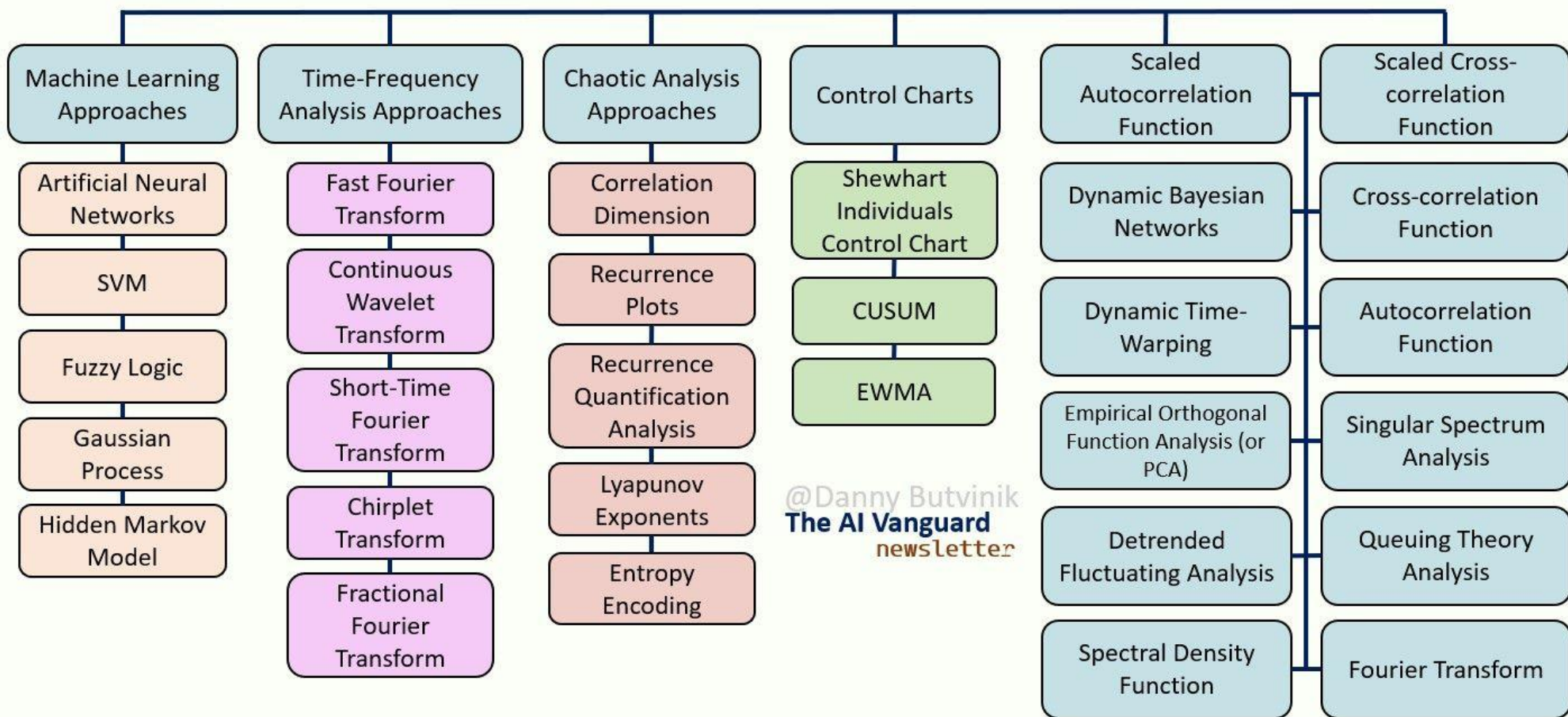
Separate model for each time series

- Naïve
- ETS
- ARIMA

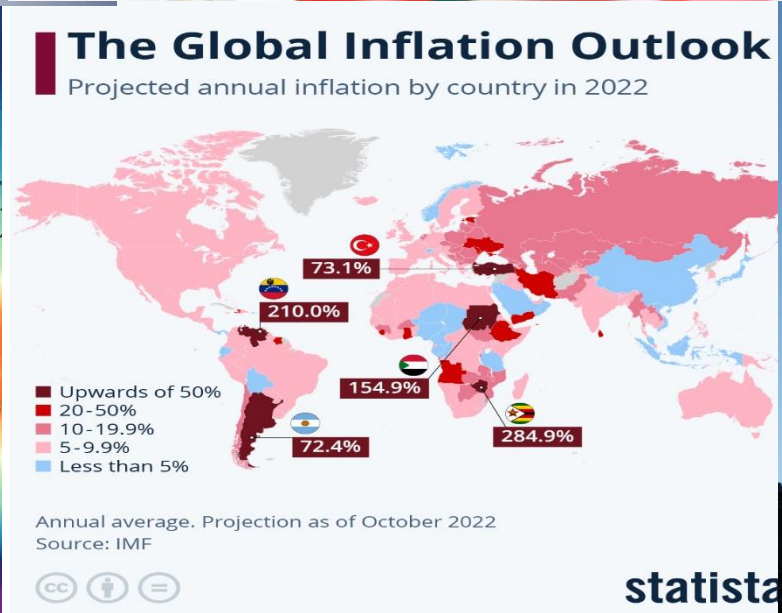
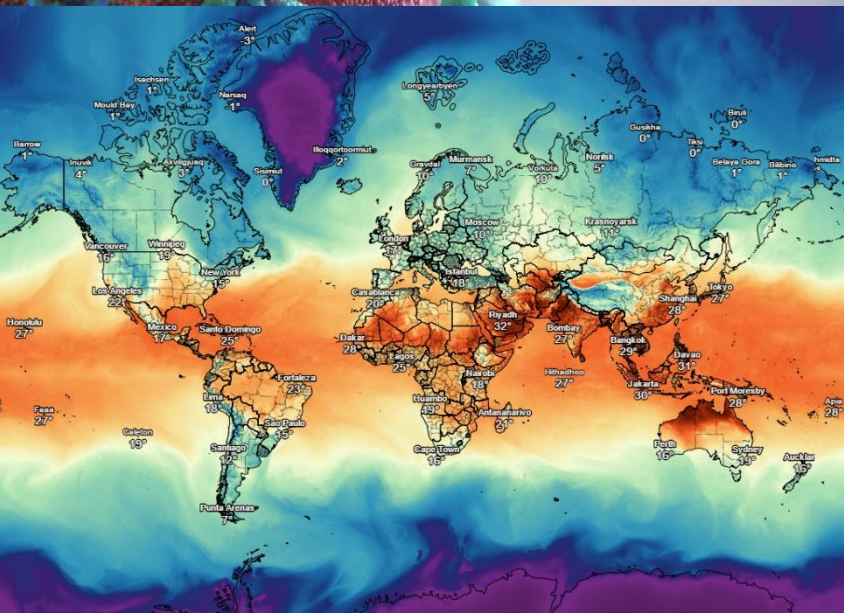
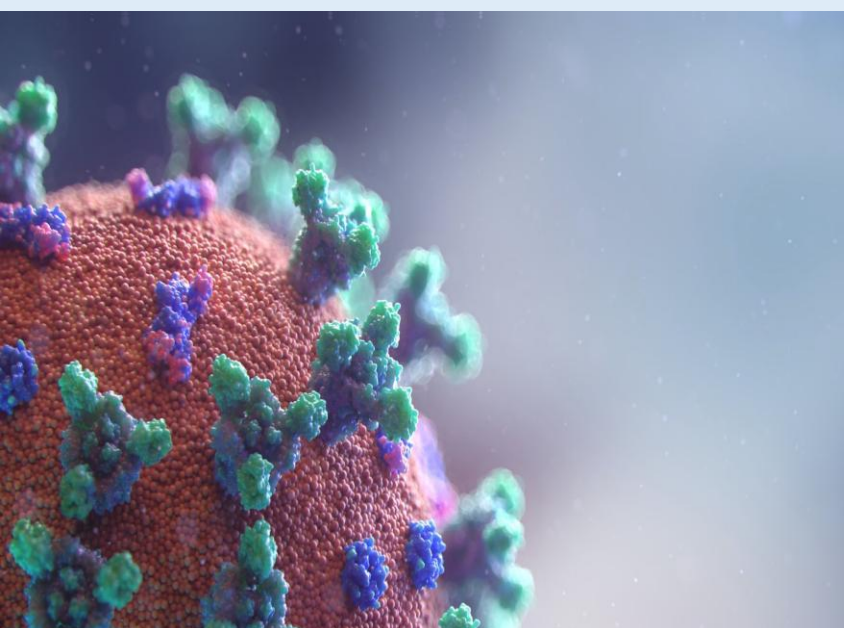
Time Series Approaches

Time series Analysis

Selected approaches



What can we forecast?



HAPPY FORECASTING

“A good forecaster is not smarter than everyone else, he merely has his ignorance better organised.”

